A Vanilla zkVM Architecture

George Kadianakis Dmitry Khovratovich Benedikt Wagner Arantxa Zapico

Ethereum Foundation Cryptography Research

September 8, 2026

This document is a reference example illustrating the expected scope and level of rigor. It is not meant to describe a zkVM that is implementable and efficient, and the described zkVM may not even be sound (we provide a best-effort security proof in Section 5 that has not been peer-reviewed and (over-)idealizes certain aspects). Submitted documents should provide additional detail where applicable.

Overview

This document describes a sample *vanilla zkVM* architecture designed according to the W1 deliverable of the EF zkVM whitepaper guidelines[1]. The architecture proves execution of a fixed RV+ (extended RISC-V) guest program using:

- Segmented execution traces
- Specialized constraint systems ("chips")
- A system of interaction buses
- In-circuit bus-membership checks for bus consistency
- Abstract knowledge-sound SNARKs for all circuits
- Recursive aggregation of segment proofs

After execution, the trace is divided into segments. Each segment is proven by four inner circuits tied together by a shared bus, whose proofs are verified by a single segment circuit. Segment proofs are then recursively aggregated in a binary tree until a single proof attests to correctness of the entire program execution.

1 Execution Model

We define how a program is executed.

[1] `https://zkevm.ethereum.foundation/blog/cryptography-research-update`

1.1 Program, Execution, and VM State

A **program** $\mathcal{P}$ is a fixed RV+ binary, compiled from an Ethereum State Transition function. Its inputs $\mathcal{I}$ are derived from the Ethereum state (*initial state*), and the execution witness $\mathcal{W}$, and its outputs $\mathcal{O}$ are the final Ethereum state (*final state*).

The program $\mathcal{P}$ operates on the **VM state** S, defined as concatenation of the following components:

- `pc`: 32-bit program counter
- `regs`$[0..k-1]$: list of k 32-bit register values
- `mem`[]: byte-addressed writable memory

We separately denote the code of the program by `code`. It is immutable and read-only, and is not part of the VM state.

A single instruction step of the program execution is defined as a transition from one VM state to another according to the semantics of the instruction op retrieved from `code` at the program counter $\texttt{pc}_1$. This is denoted as follows:

$$S := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \xrightarrow{\text{op:=}\texttt{code}[\texttt{pc}_1]} S' := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2) \tag{1}$$

For example, if op is an addition instruction, which adds the first two registers and stores in the third register, then the semantics of the step is defined as

$$\texttt{pc}_2 = \texttt{pc}_1 + 1, \tag{2}$$

$$\texttt{regs}_2 = \texttt{regs}_1 \text{ except that } \texttt{regs}_2[2] := \texttt{regs}_1[0] + \texttt{regs}_1[1], \tag{3}$$

$$\texttt{mem}_2 = \texttt{mem}_1. \tag{4}$$

We will use the notation $(\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1) \to (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ to denote that S transitions to S' by executing instruction $\text{op} := \texttt{code}[\texttt{pc}_1]$.

An **execution trace** for program `code` applied to inputs $\mathcal{I}$ is a sequence of VM states

$$\mathcal{T}: \quad S_0 \to S_1 \to \cdots \to S_T$$

where each step corresponds to execution of one instruction.

A **zkVM proof** for $(\mathcal{P}, \mathbb{E}, \mathbb{E}')$, where $\mathbb{E}$ and $\mathbb{E}'$ are the initial and final Ethereum states, is a SNARK that proves the following:

$$\begin{cases} S_0 \to \cdots \to S_T; \\ S_0 := (\texttt{pc}_0 := 0, \texttt{regs}, \texttt{mem}) \text{ where } \texttt{regs}, \texttt{mem} \text{ are derived from } \mathbb{E}; \\ S_T := (\texttt{pc}_T, \texttt{regs}', \texttt{mem}') \text{ where } \texttt{regs}', \texttt{mem}' \text{ are derived from } \mathbb{E}'; \end{cases}$$

1.2 Supported Instructions

The zkVM supports the RV+ instruction set, which is an extension of the RV32IM instruction set. Concretely, all operations:

- conform to the state transition (1), i.e. transform $(\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ to $(\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ according to the semantics of the instruction op retrieved from `code` at the program counter $\texttt{pc}_1$.

- are all standard RISC-V instructions[2] (executed according to the standard RISC-V semantics) or additional instructions (precompiles) for efficient execution of certain operations — concretely, Keccak and Poseidon.

For our purpose the exact semantics of the instructions is not important, and we treat them as black boxes with their own predicates modeling the correctness of the execution.

Concretely, for given instruction *op*, the predicate φ_{op} has the semantics

$$\varphi_{op}(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1, \mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) = \text{TRUE} \iff (\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1) \xrightarrow{\text{op}} (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) \wedge \mathtt{code}[\mathtt{pc}_1] = op \quad (5)$$

For instance, if *op* is an ADD instruction, then $\mathtt{mem}_1 = \mathtt{mem}_2$, $\mathtt{pc}_1 + 1 = \mathtt{pc}_2$ and the registers change according to the ADD instruction.

Exceptions are memory operations, which we assume have the following syntax and semantics:

$$\begin{aligned} \text{read} : (\mathtt{regs}_1 := (addr, *, \ldots), \mathtt{mem}) &\to (\mathtt{regs}_2 := (addr, x, \ldots), \mathtt{mem}), \\ \mathtt{mem}[addr] &= x, \end{aligned} \quad (6)$$

$$\begin{aligned} \text{write} : (\mathtt{regs}_1 := (addr, x, \ldots), \mathtt{mem}) &\to (\mathtt{regs}_2 := (addr, x, \ldots), \mathtt{mem}'), \\ \mathtt{mem}'[i] &= \begin{cases} x & \text{if } i = addr, \\ \mathtt{mem}[i] & \text{otherwise.} \end{cases} \end{aligned} \quad (7)$$

We decompose each memory predicate into a pc-and-register component (independent of memory) and an explicit memory-access equation:

$$\varphi_{\text{read}}(S_1, S_2) := \varphi'_{\text{read}}(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{pc}_2, \mathtt{regs}_2) \ \wedge \ \mathtt{mem}_1[\mathtt{regs}_1[0]] = \mathtt{regs}_2[1] \ \wedge \ \mathtt{mem}_2 = \mathtt{mem}_1 \quad (8)$$

$$\varphi_{\text{write}}(S_1, S_2) := \varphi'_{\text{write}}(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{pc}_2, \mathtt{regs}_2) \ \wedge \ \mathtt{mem}_2 = \mathtt{mem}_1[\mathtt{regs}_1[0] \mapsto \mathtt{regs}_1[1]] \quad (9)$$

where $\mathtt{regs}_1[0]$ denotes the address register, $\mathtt{regs}_1[1]$ (resp. $\mathtt{regs}_2[1]$) the value register before (resp. after) the step, and φ'_{read}, φ'_{write} depend only on $\mathtt{pc}$ and $\mathtt{regs}$. Both φ'_{read} and φ'_{write} include the well-formedness conjunct $\mathtt{regs}_1[0] \in [n]$, where n is the number of addressable memory cells: without it the expressions $\mathtt{mem}_1[\mathtt{regs}_1[0]]$ and $\mathtt{mem}_1[\mathtt{regs}_1[0] \mapsto \mathtt{regs}_1[1]]$ above would be undefined. The conjunct is memory-free, so it is inherited verbatim by the committed-memory versions of the two predicates, and it is what lets the security proof treat $\mathtt{regs}_1[0]$ as a legal index of the committed vector.

Every pc-and-register part φ'_{op} — of memory and non-memory operations alike — also carries the fetch conjunct $\mathtt{code}[\mathtt{pc}_1] = op$ of (5); the conjunct depends only on $\mathtt{pc}_1$ (the code $\mathtt{code}$ is fixed), so it is memory-free and belongs in φ'_{op}. This placement is load-bearing: the security analysis (Remark 14) relies on it to conclude that at most one operation's disjunct of the step predicate can hold, and that this operation is $\mathtt{code}[\mathtt{pc}_1]$.

Teams whose architecture selects the operation differently (e.g. via selector columns) must ensure an equivalent guarantee: the constraint system must force the satisfied operation branch to be the one fetched at $\mathtt{pc}_1$. Without it, the memory-reconstruction rule of Proposition 13 can diverge from the satisfied branch and the commitment invariant fails without any binding violation.

[2] `https://riscv.org/technical/specifications/`

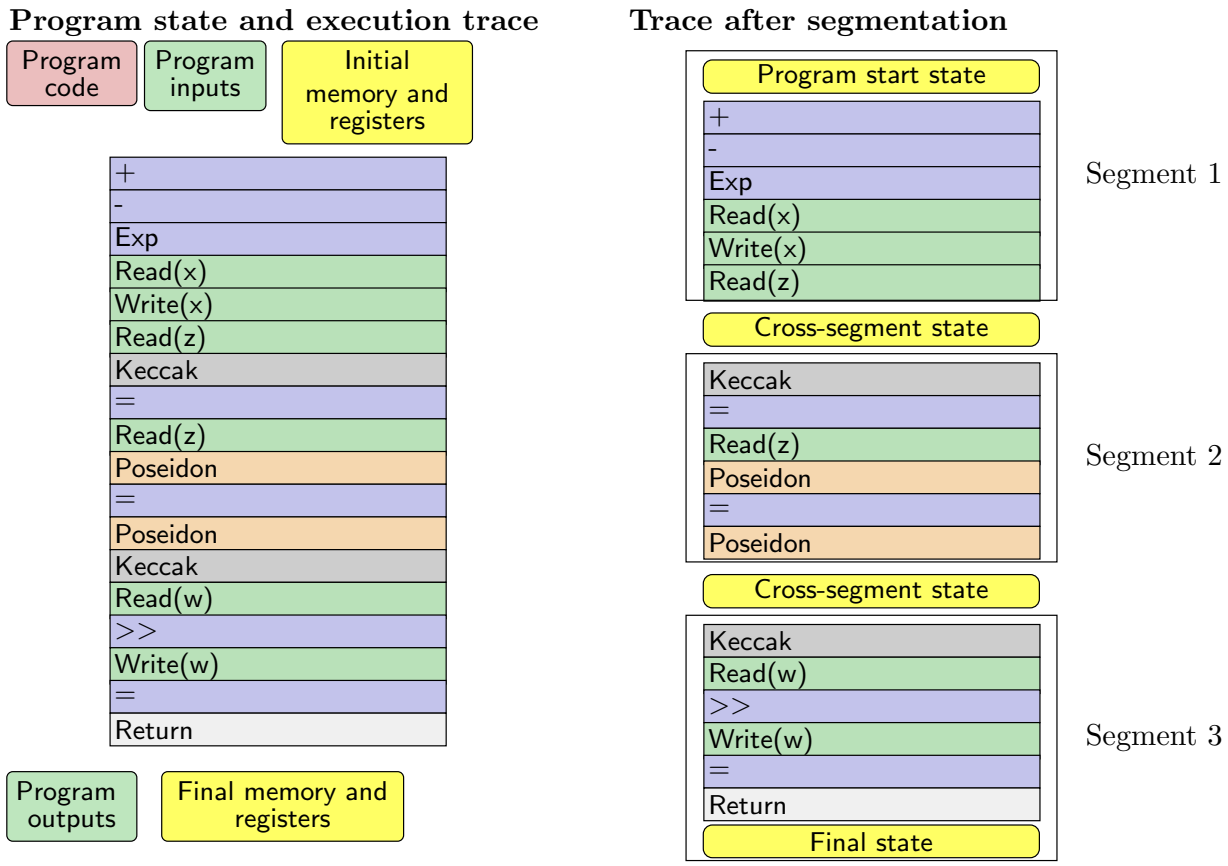

Figure 1: Segmentation of the execution trace into 3 segments

The conjunct $\texttt{mem}_2 = \texttt{mem}_1$ in (8) is kept explicit for the same reason as for the non-memory operations: a read must not be able to silently alter memory. Without it φ_{read} would leave $\texttt{mem}_2$ completely unconstrained, and a trace satisfying φ_{step} at every step could still replace the whole memory at a read. Note that φ_{write} needs no such conjunct, as (9) already determines $\texttt{mem}_2$ entirely.

2 Segmentation

We now explain how execution is split into segments.

2.1 Execution Segments

After execution, the execution trace $\mathcal{T}$ is divided into segments $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_m$ of exactly N_{seg} instructions[3] each (cf. Fig. 1). Each segment $\mathcal{G}_i$ corresponds to a contiguous subsequence of the execution trace, from S_{start_i} to S_{end_i}, and is proven by a set of separate proofs. The segmentation of the execution trace into consecutive segments allows reducing the proof of the entire execution to proving the validity of each segment and consistency between them. The global consistency argument asserts that the segments are consistent with each other and with the public inputs/outputs:

$$S_{end_i} = S_{start_{i+1}} \text{ for all } i \in [m-1], \quad S_{start_1} = S_0, \quad S_{end_m} = S_T.$$

[3]The instruction set has a no-op instruction to pad the segment to the full length. The parameter N_{seg} is assumed to be hardcoded and fixed, and known to the verifier.

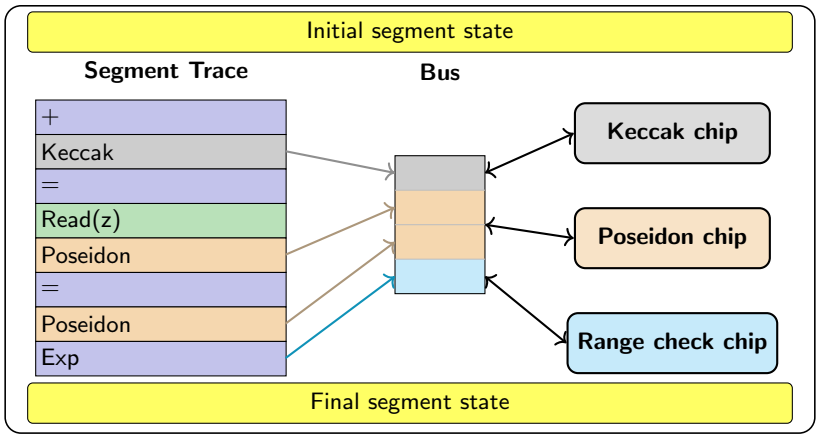

Figure 2: Segment structure: segment trace connects to the bus which then connects to chips.

2.2 Segment layout

Each segment is represented in an equivalent form as a collection of one trace and several *chips*, listed below (cf. Fig. 2).

Segment trace It consists of all operations of the execution trace within the segment and state variables, including the program counter, register values, memory state, etc. It is a collection of N_{seg} rows, each corresponding to one instruction step from the segment. The segment trace does not enforce the semantics of all instructions, but only a subset of them, and the rest of the instructions are proven in separate chips and their consistency with the segment trace is enforced by *consistency arguments*.

Keccak chip It consists of all Keccak precompile calls in the segment, and their inputs/outputs.

Poseidon chip It consists of all Poseidon precompile calls in the segment, and their inputs/outputs.

Range check chip It consists of all range check statements occurring in the segment.

Local consistency argument

Teams should describe how their bus consistency argument works in terms of lookups used and how they integrate into the overall proof system.

The consistency arguments enforce that the operations proven in the chips are consistent with the segment trace. This is done using the concept of a *bus* connecting the segment trace and the chips, and lookup arguments to enforce consistency of the bus values with the segment trace and the chip traces.

3 Correct execution of a program

In this section, we define via predicates what it means that a program is correctly executed. We will use these predicates in Section 4 to define the relations that the zkVM proves.

3.1 Predicates for individual operations

Teams should list all operations supported by their zkVM and define the corresponding predicates based on their architecture.

Each operation is treated as a black box with its own predicate φ_{op} (cf. (5)). Throughout this section, subscripts 1 and 2 denote states before and after an operation. We also denote $S_1 := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ and $S_2 := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ as the states before and after the operation.

We group operations as follows:

- **Memory operations.** Read: $\varphi_{\text{read}}(S_1, S_2)$ — does not modify memory. Write: $\varphi_{\text{write}}(S_1, S_2)$ — may modify memory.
- **Precompiles.** Keccak: $\varphi_{\text{keccak}}(S_1, S_2)$. Poseidon: $\varphi_{\text{poseidon}}(S_1, S_2)$.
- **Arithmetic without range checks.** $\text{op}_1, \ldots, \text{op}_{10}$ with predicates $\varphi_{\text{op}_i}(S_1, S_2)$.
- **Arithmetic with range checks.** $\text{op}_{11}, \ldots, \text{op}_{20}$ with predicates $\varphi_{\text{op}_i}(S_1, S_2)$, each decomposable as $\varphi'_{\text{op}_i}(S_1, S_2) \wedge \varphi_{\text{range}}(S_1)$ where the constants defining the range check are always in the last two registers.

All operations except read and write neither read nor write memory. For such an operation we take $\varphi_{op}(S_1, S_2) := \varphi'_{op}(\texttt{pc}_1, \texttt{regs}_1, \texttt{pc}_2, \texttt{regs}_2) \wedge \texttt{mem}_2 = \texttt{mem}_1$, where φ'_{op} depends only on `pc` and `regs` and, as for every operation, includes the fetch conjunct $\texttt{code}[\texttt{pc}_1] = op$ of (5); the memory-equality conjunct $\texttt{mem}_2 = \texttt{mem}_1$ is kept explicit so that a non-memory step cannot silently alter memory. We write φ'_{op} (and its committed version) for the pc-and-register part and keep the conjunct explicit throughout.

3.2 Correct execution of a single step

Teams should define the step predicate for their architecture, specifying how the correct operation is selected and verified at each step.

A single step executes the instruction at the current program counter. The step predicate selects the matching operation via a disjunction over all opcodes:

$$\varphi_{step}(S_1, S_2) := \bigvee_{op \in \text{operations}} \varphi_{op}(S_1, S_2)$$

where $S_1 := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ and $S_2 := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ are the states before and after the step. Recall from (5) that each φ_{op} includes the fetch conjunct $\texttt{code}[\texttt{pc}_1] = op$; hence φ_{step} binds every step to the fixed program `code`.

3.3 Delegating expensive operations using a Bus

Teams should describe how expensive operations (e.g., precompiles, range checks) are delegated to a bus or similar mechanism in their architecture.

Range checks and precompile calls are expensive to prove inline. Instead, we collect them in a shared *bus* $\mathcal{B}$ and verify separately, proving the consistency along the way. The *bus* $\mathcal{B}$ is a collection

of all precompile calls and range checks within a segment. That is, it has the following structure:

$$\mathcal{B} = \{(\underbrace{\text{op}^{(i)}}_{\in\{\text{keccak,poseidon}\}}, S_1^{(i)}, S_2^{(i)})\}_{i=1}^{b} \cup \{(\text{range}, S^{(j)})\}_{j=1}^{r}$$

where b is the number of precompile calls and r is the number of range checks.

Now we define auxiliary predicates that filter the bus by operation type and assert correctness of each entry: $\varphi_{\text{keccak}}(\mathcal{B})$ checks all Keccak entries, $\varphi_{\text{poseidon}}(\mathcal{B})$ checks all Poseidon entries, and $\varphi_{\text{range}}(\mathcal{B})$ checks all range-check entries. Concretely:

$$\begin{aligned}
\varphi_{\text{keccak}}(\mathcal{B}) &:= \bigwedge_{(\text{keccak},S_1,S_2)\in\mathcal{B}} \varphi_{\text{keccak}}(S_1, S_2) \\
\varphi_{\text{poseidon}}(\mathcal{B}) &:= \bigwedge_{(\text{poseidon},S_1,S_2)\in\mathcal{B}} \varphi_{\text{poseidon}}(S_1, S_2) \\
\varphi_{\text{range}}(\mathcal{B}) &:= \bigwedge_{(\text{range},S)\in\mathcal{B}} \varphi_{\text{range}}(S)
\end{aligned}$$

We now rewrite the step predicate (3.2) to separate inline verification from bus-deferred verification. The new form is equivalent[4]:

$$\varphi_{\text{step}}(S_1, S_2, \mathcal{B}) := \varphi_{\text{step,bus}}(S_1, S_2, \mathcal{B}) \wedge \varphi_{\text{keccak}}(\mathcal{B}) \wedge \varphi_{\text{poseidon}}(\mathcal{B}) \wedge \varphi_{\text{range}}(\mathcal{B}) \tag{10}$$

where

$$\begin{aligned}
\varphi_{\text{step,bus}}(S_1, S_2, \mathcal{B}) := & \bigvee_{op\in\{\text{read},\text{write},\text{op}_1,\ldots,\text{op}_{10}\}} (\varphi_{\text{op}}(S_1, S_2)) \\
& \bigvee_{op\in\{\text{op}_{11},\ldots,\text{op}_{20}\}} \left(\varphi'_{\text{op}}(S_1, S_2) \wedge (\text{range}, S_1) \in \mathcal{B} \wedge \texttt{mem}_2 = \texttt{mem}_1\right) \\
& \bigvee_{op\in\{\text{Keccak},\text{Poseidon}\}} ((op, S_1, S_2) \in \mathcal{B} \wedge \texttt{mem}_2 = \texttt{mem}_1)
\end{aligned} \tag{11}$$

Recall that $S_1 := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ and $S_2 := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$. The predicate $\varphi_{\text{step,bus}}$ separates operations into three categories:

- **Memory ops and arithmetic without range checks** $(\text{read}, \text{write}, \text{op}_1, \ldots, \text{op}_{10})$ are verified inline via their predicates.
- **Arithmetic with range checks** $(\text{op}_{11}, \ldots, \text{op}_{20})$: the arithmetic logic φ_{op} is verified inline, but the range check is deferred by recording (range, S_1) in $\mathcal{B}$.
- **Precompiles** (Keccak, Poseidon) are fully deferred — the step only checks that the call appears in $\mathcal{B}$, while correctness will be verified by $\varphi_{\text{keccak}}(\mathcal{B})$, $\varphi_{\text{poseidon}}(\mathcal{B})$ in (10).

[4]Formally the new predicate is even stronger as it also asserts the correctness of the operations in $\mathcal{B}$ not listed in the segment. We omit checking for such operations for efficiency.

3.4 Committed memory

Teams should describe how memory is committed and how memory operations are verified in their architecture.

The full memory state is too large to pass around directly, so we replace it with the Merkle tree commitment Com. Concretely, Com := (Commit, Open, Verify) is the Merkle tree vector commitment scheme: Commit(mem) returns the Merkle root of the memory vector mem; Open(mem, i) returns the authentication path for position i; and Verify$(C, i, v, \pi) = 1$ iff π is a valid Merkle authentication path certifying that position i of the committed vector holds value v under root C. The commitment scheme is used for the following memory operations: read (read), write (write), and any other operation that reads or writes a memory cell. We define a *state with committed memory* $\hat{S} := (\mathtt{pc}, \mathtt{regs}, \hat{\mathtt{mem}})$, where $\hat{\mathtt{mem}} := \mathsf{Commit}(\mathtt{mem})$ is the memory commitment value, replacing the full memory in $S := (\mathtt{pc}, \mathtt{regs}, \mathtt{mem})$ with its commitment.

The opening proofs π (for reads) and $(\pi_{\text{write}}, v_{\mathsf{old}})$ (for writes) are passed as explicit arguments to the predicates rather than existentially quantified. This is essential for the security proof: the binding properties of $\text{Com}_{\mathsf{mem}}$ apply only to proofs that are efficiently produced by a PPT algorithm, so the proofs must appear as concrete witness fields in $\mathcal{R}_{0,\text{step}}$ from which the extractor can retrieve them, rather than merely existing abstractly.

The read operation predicate becomes

$$\begin{aligned}\widehat{\varphi}_{\text{read}}(\hat{S}_1, \hat{S}_2, \pi) \;:=\; & \varphi'_{\text{read}}(\hat{S}_1.\mathtt{pc}, \hat{S}_1.\mathtt{regs}, \hat{S}_2.\mathtt{pc}, \hat{S}_2.\mathtt{regs}) \\ & \wedge\; \hat{S}_1.\mathtt{mem} = \hat{S}_2.\mathtt{mem} \\ & \wedge\; \mathsf{Verify}(\hat{S}_1.\mathtt{mem},\; \hat{S}_1.\mathtt{regs}[0],\; \hat{S}_2.\mathtt{regs}[1],\; \pi) = 1.\end{aligned} \tag{12}$$

Here Verify is the Merkle opening-verification algorithm described above; the proof π is supplied explicitly as a witness by the prover. The write operation predicate becomes

$$\begin{aligned}\widehat{\varphi}_{\text{write}}(\hat{S}_1, \hat{S}_2, \pi^{\mathsf{mem}}) \;:=\; & \varphi'_{\text{write}}(\hat{S}_1.\mathtt{pc}, \hat{S}_1.\mathtt{regs}, \hat{S}_2.\mathtt{pc}, \hat{S}_2.\mathtt{regs}) \\ & \wedge\; \mathsf{Verify}(\hat{S}_1.\mathtt{mem},\; \hat{S}_1.\mathtt{regs}[0],\; \pi^{\mathsf{mem}}.v_{\mathsf{old}},\; \pi^{\mathsf{mem}}.\pi_{\text{write}}) = 1 \\ & \wedge\; \mathsf{Verify}(\hat{S}_2.\mathtt{mem},\; \hat{S}_1.\mathtt{regs}[0],\; \hat{S}_1.\mathtt{regs}[1],\; \pi^{\mathsf{mem}}.\pi_{\text{write}}) = 1.\end{aligned} \tag{13}$$

Here, for a write step, $\pi^{\mathsf{mem}} = (\pi_{\text{write}}, v_{\mathsf{old}})$ bundles the Merkle authentication path π_{write} together with the old value v_{old} stored at the written address before the step.

Replacing memory with commitments in (10) gives the step relation:

$$\widehat{\varphi}_{step}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}, \pi^{\mathsf{mem}}) := \widehat{\varphi}_{step,bus}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}, \pi^{\mathsf{mem}}) \wedge \widehat{\varphi}_{\text{keccak}}(\widehat{\mathcal{B}}) \wedge \widehat{\varphi}_{\text{poseidon}}(\widehat{\mathcal{B}}) \wedge \widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) \tag{14}$$

where $\widehat{\mathcal{B}}$ replaces the memory entries with committed memory (which is practically invisible as the bus operations do not use memory); $\widehat{\varphi}_{\text{keccak}}$, $\widehat{\varphi}_{\text{poseidon}}$, $\widehat{\varphi}_{\text{range}}$ are obtained from φ_{keccak}, $\varphi_{\text{poseidon}}$, φ_{range} by the same substitution. Their pc-and-register parts φ'_{op} are literally unchanged, since those parts do not reference memory; the only difference is that the conjunct $\mathtt{mem}_2 = \mathtt{mem}_1$ carried by every non-memory operation predicate becomes the commitment equality $\hat{S}_1.\mathtt{mem} = \hat{S}_2.\mathtt{mem}$. (The range-check predicate $\varphi_{\text{range}}(S_1)$ constrains registers only and has no memory conjunct at all, so it is unchanged outright.) Finally, $\widehat{\varphi}_{step,bus}$ is defined analogously to $\varphi_{\text{step,bus}}$ but with all operation

predicates replaced by their committed-memory versions:

$$\begin{aligned}\widehat{\varphi}_{step,bus}(\hat{S}_1,\hat{S}_2,\widehat{\mathcal{B}},\pi^{\mathsf{mem}}) := \widehat{\varphi}_{\text{read}}(\hat{S}_1,\hat{S}_2,\pi^{\mathsf{mem}}) \;\vee\; \widehat{\varphi}_{\text{write}}(\hat{S}_1,\hat{S}_2,\pi^{\mathsf{mem}}) \\ \vee \bigvee_{op\in\{\text{op}_1,\text{op}_2,\ldots,\text{op}_{10}\}} \Big(\widehat{\varphi}_{op}(\hat{S}_1,\hat{S}_2) \wedge \hat{S}_1.\mathtt{mem} = \hat{S}_2.\mathtt{mem}\Big) \\ \vee \bigvee_{op\in\{\text{op}_{11},\text{op}_{12},\ldots,\text{op}_{20}\}} \Big(\widehat{\varphi}'_{op}(\hat{S}_1,\hat{S}_2) \wedge (\text{range},\hat{S}_1) \in \widehat{\mathcal{B}} \wedge \hat{S}_1.\mathtt{mem} = \hat{S}_2.\mathtt{mem}\Big) \\ \vee \bigvee_{op\in\{\text{Keccak},\text{Poseidon}\}} \Big((op,\hat{S}_1,\hat{S}_2) \in \widehat{\mathcal{B}} \wedge \hat{S}_1.\mathtt{mem} = \hat{S}_2.\mathtt{mem}\Big) \quad (15)\end{aligned}$$

where, for a read step, π^{mem} is the Merkle authentication path π; for a write step, $\pi^{\mathsf{mem}} = (\pi_{\text{write}}, v_{\mathsf{old}})$ is the pair described above; for all other steps the memory-proof argument is unused.

Note that (10) and (14) are not equivalent, and in fact are not even computationally equivalent: satisfying the committed-memory predicate does not imply satisfying the full-memory predicate even for computationally bounded provers, because opening a commitment to the full memory vector may be infeasible. Proposition 13 shows that the gap is bounded by the position-binding and update-binding advantages of the commitment scheme $\text{Com}_{\mathsf{mem}}$.

4 Proof Architecture

We now define relations that are proven using individual SNARKs, and that recursively combine into the final proof.

4.1 Topology

The proof pipeline consists of multiple circuit types.

Teams should add footnotes to each circuit subsection below linking to the corresponding soundcalc entry for that circuit's parameters and security analysis.

Below we expose some aggregate information.

Circuit	**Purpose**	Relations
Inner segment circuits (leaf layer):		
`inner-step`	Segment trace	$\mathcal{R}_{0,\text{step}}$
`inner-keccak`	Keccak precompile	$\mathcal{R}_{0,\text{keccak}}$
`inner-poseidon`	Poseidon precompile	$\mathcal{R}_{0,\text{poseidon}}$
`inner-range`	Range checks	$\mathcal{R}_{0,\text{range}}$
`segment`	Segment proof	$\mathcal{R}_1$
`convert`	1-to-1 normalization	$\mathcal{R}_2$
`combine`	2-to-1 merge	$\mathcal{R}_3$
`embed`	final proof	$\mathcal{R}_4$

- *Inner segment circuits (leaf layer).* These four circuits jointly prove one segment; the `segment` circuit below verifies all four.
  - `inner-step`: Proves the segment trace execution (with precompiles and range checks excluded).

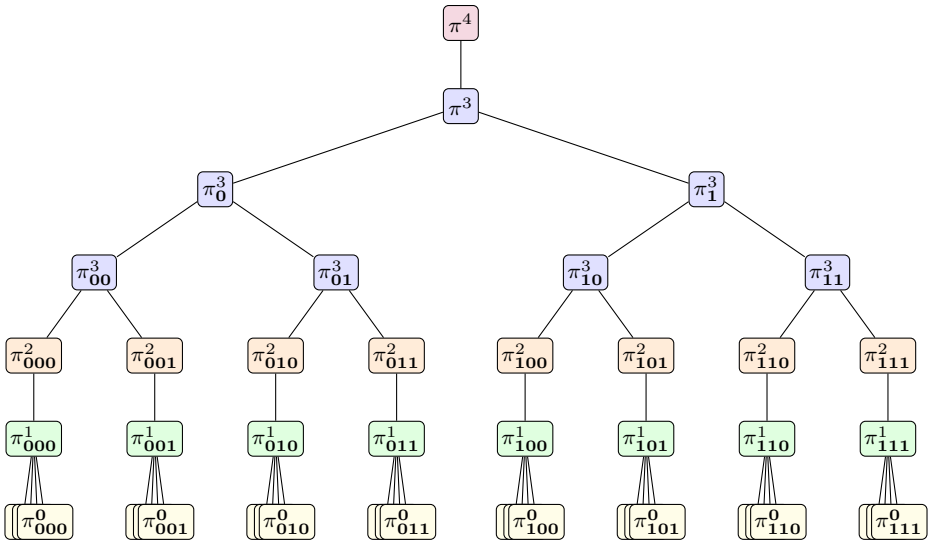

Figure 3: Recursion topology. Colors: `inner`, `segment`, `convert`, `combine`, `embed`.

  - `inner-keccak`: Proves the Keccak precompile.
  - `inner-poseidon`: Proves the Poseidon precompile.
  - `inner-range`: Proves the range checks.

- `segment`: Verifies the four inner proofs, effectively proving one segment
- `convert`: A 1-to-1 recursion circuit that verifies a single `segment` proof.
- `combine`: A recursive 2-to-1 circuit that verifies two child proofs, each of which may be either a `convert` or a `combine` proof.
- `embed`: The circuit that produces the final proof.

We visualize the recursion topology in Fig. 3. Here, $\pi^{\mathbf{0}}$ denotes the four inner proofs (one each for $\mathcal{R}_{0,\text{step}}$, $\mathcal{R}_{0,\text{keccak}}$, $\mathcal{R}_{0,\text{poseidon}}$, $\mathcal{R}_{0,\text{range}}$), π^1 are proofs of $\mathcal{R}_1$ and thus the `segment` circuit. Similarly, π^2 proves the `convert` circuit, π^3 the `combine` circuit, and π^4 the `embed` circuit.

4.2 `inner` circuits

To optimize the proof size and prover time, we introduce inner segment circuits that prove the correctness of the segment trace, of precompiles, and of range checks separately. As those circuits share the same bus, we commit to the latter to make the relations succinct. These circuits are then recursively combined to prove the correctness of the full segment.

The first relation $\mathcal{R}_{0,\text{step}}$ chains N_{seg} consecutive valid steps from $\hat{S}_{in}$ to $\hat{S}_{out}$, with the bus committed in the public input. The remaining three relations each assert correctness of all entries of a given type in the bus (Keccak, Poseidon, or range checks respectively), again with the bus

committed[5] in the public input.

$$\mathcal{R}_{0,\text{step}} = \left(\left(\begin{array}{l} (\hat{S}_{in}, \hat{S}_{out}, C_{\hat{\mathcal{B}}}); \\ (\widehat{\mathcal{B}}, \{\hat{S}_i\}_{1\le i\le N_{\text{seg}}+1}, \{\pi_i^{\mathsf{mem}}\}_{1\le i\le N_{\text{seg}}}) \end{array} \right) \;\middle|\; \begin{array}{c} \hat{S}_1 = \hat{S}_{in} \\ \hat{S}_{N_{\text{seg}}+1} = \hat{S}_{out} \\ \bigwedge_{i=1}^{N_{\text{seg}}} \widehat{\varphi}_{step,bus}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi_i^{\mathsf{mem}}) = \text{TRUE} \\ C_{\hat{\mathcal{B}}} = \text{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) \end{array} \right) \tag{16}$$

where π_i^{mem} is the memory-opening witness for step i: for a read step it is the Merkle authentication path π required by $\widehat{\varphi}_{\text{read}}$ (12); for a write step it is the pair $\pi_i^{\mathsf{mem}} = (\pi_{\text{write}}, v_{\mathsf{old}})$ required by $\widehat{\varphi}_{\text{write}}$ (13); for all other steps it is unused. The program code `code` is hard-wired into the `inner-step` circuit — it appears in the fetch conjunct $\texttt{code}[\texttt{pc}_1] = op$ of each operation predicate via (5) — so the `inner-step` verification key binds the proof to the fixed program.

$$\mathcal{R}_{0,\text{keccak}} = \left(\left(C_{\hat{\mathcal{B}}};\ \widehat{\mathcal{B}} \right) \;\middle|\; \begin{array}{c} \widehat{\varphi}_{\text{keccak}}(\widehat{\mathcal{B}}) = \text{TRUE} \\ C_{\hat{\mathcal{B}}} = \text{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) \end{array} \right) \tag{17}$$

$$\mathcal{R}_{0,\text{poseidon}} = \left(\left(C_{\hat{\mathcal{B}}};\ \widehat{\mathcal{B}} \right) \;\middle|\; \begin{array}{c} \widehat{\varphi}_{\text{poseidon}}(\widehat{\mathcal{B}}) = \text{TRUE} \\ C_{\hat{\mathcal{B}}} = \text{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) \end{array} \right) \tag{18}$$

$$\mathcal{R}_{0,\text{range}} = \left(\left(C_{\hat{\mathcal{B}}};\ \widehat{\mathcal{B}} \right) \;\middle|\; \begin{array}{c} \widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) = \text{TRUE} \\ C_{\hat{\mathcal{B}}} = \text{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) \end{array} \right) \tag{19}$$

The circuits are called, respectively, `inner-step`, `inner-keccak`, `inner-poseidon`, and `inner-range`.

Note that our writeup is a bit informal here, as we never formally defined Com. Teams should do that, as the security analysis later will require a specific binding notion of Com.

4.3 `segment` circuit

The `segment` circuit recursively verifies the four inner proofs, tying them together via a shared bus commitment $C_{\hat{\mathcal{B}}}$. Let $\Pi_{0,j} = (\Pi_{0,j}.\mathsf{Prove}, \Pi_{0,j}.\mathsf{Verify})$ be the SNARK for $\mathcal{R}_{0,j}$, where $j \in \{\text{step}, \text{keccak}, \text{poseidon}, \text{range}\}$.

$$\mathcal{R}_1 = \left\{ (\hat{S}_{in}, \hat{S}_{out}); (C_{\hat{\mathcal{B}}}, \pi_{\text{step}}, \pi_{\text{keccak}}, \pi_{\text{poseidon}}, \pi_{\text{range}}) \;\middle|\; \begin{array}{c} \Pi_{0,\text{step}}.\mathsf{Verify}((\hat{S}_{in},\ \hat{S}_{out},\ C_{\hat{\mathcal{B}}}), \pi_{\text{step}}) = 1 \\ \Pi_{0,\text{keccak}}.\mathsf{Verify}(C_{\hat{\mathcal{B}}}, \pi_{\text{keccak}}) = 1 \\ \Pi_{0,\text{poseidon}}.\mathsf{Verify}(C_{\hat{\mathcal{B}}}, \pi_{\text{poseidon}}) = 1 \\ \Pi_{0,\text{range}}.\mathsf{Verify}(C_{\hat{\mathcal{B}}}, \pi_{\text{range}}) = 1 \end{array} \right\}$$

4.4 `convert` circuit

The `convert` circuit is a 1-to-1 recursion circuit that consumes a proof of $\mathcal{R}_1$ and verifies it. Its public inputs are the boundary states and the segment size; its witness is the proof π_1. To formally define the relation, let $\Pi_1 = (\Pi_1.\mathsf{Prove}, \Pi_1.\mathsf{Verify})$ be the SNARK proving protocol for relation $\mathcal{R}_1$.

$$\mathcal{R}_2 = \left\{ \ ((\hat{S}_0,\ \hat{S}_N,\ N_{\text{seg}});\ \pi_1) \;\middle|\; \Pi_1.\mathsf{Verify}((\hat{S}_0,\ \hat{S}_N), \pi_1) = 1 \ \right\}$$

Note that triples for which the last component is not N_{seg} are never in $\mathcal{R}_2$.

[5] Without loss of generality we assume that the same commitment scheme is used for the bus.

4.5 `combine` circuit

The `combine` circuit is a recursive 2-to-1 circuit that verifies two child proofs, each of which may be either a `convert` proof or another `combine` proof. It checks three things:

- Each child proof is valid under $\Pi_2.\mathsf{Verify}$ if the child covers exactly one segment ($N_{child} = N_{\text{seg}}$), or under $\Pi_3.\mathsf{Verify}$ if it covers at least two ($N_{child} \geq 2N_{\text{seg}}$), where $\Pi_i = (\Pi_i.\mathsf{Prove}, \Pi_i.\mathsf{Verify})$ is the SNARK proving protocol for relation $\mathcal{R}_i$.
- The step counts satisfy $N_L + N_R = N$, with $N_L \geq N_{\text{seg}}$, $N_R \geq N_{\text{seg}}$, and both N_L, N_R divisible by N_{seg}, to ensure well-founded recursion and uniform segment structure.
- State continuity: the witness $\hat{S}_N^L$ serves as both the final state of the left proof and the initial state of the right proof.

Since `combine` is self-recursive, repeated binary application naturally reduces any number of segment proofs into a single proof π_3 for the full execution.

For brevity, we write $\mathsf{V}_i(\mathsf{x}, \pi) := \Pi_i.\mathsf{Verify}(\mathsf{x}, \pi)$.

$$\mathcal{R}_3 = \left\{ ((\hat{S}_0^L,\, \hat{S}_N^R,\, N);\ (\pi^L,\, \pi^R,\, \hat{S}_N^L, N_L, N_R)) \;\middle|\; \begin{array}{c} \big(N_L = N_{\text{seg}} \wedge \mathsf{V}_2((\hat{S}_0^L, \hat{S}_N^L, N_L),\, \pi^L) = 1\big) \\ \vee \big(N_L \geq 2N_{\text{seg}} \wedge \mathsf{V}_3((\hat{S}_0^L, \hat{S}_N^L, N_L),\, \pi^L) = 1\big) \\ \big(N_R = N_{\text{seg}} \wedge \mathsf{V}_2((\hat{S}_N^L, \hat{S}_N^R, N_R),\, \pi^R) = 1\big) \\ \vee \big(N_R \geq 2N_{\text{seg}} \wedge \mathsf{V}_3((\hat{S}_N^L, \hat{S}_N^R, N_R),\, \pi^R) = 1\big) \\ N_L + N_R = N \\ N_{\text{seg}} \mid N_L,\ N_{\text{seg}} \mid N_R \end{array} \right\} \tag{20}$$

where $\hat{S}_0^L$ and $\hat{S}_N^R$ are the public boundary states of the combined execution, π^L and π^R are the two child proofs (each either a `convert` or `combine` proof), and $\hat{S}_N^L$ is the intermediate boundary state provided as witness. The step count decides the child's type: a child covering exactly one segment must carry a `convert` proof, a larger child a `combine` proof. This typing is what entitles the extraction argument (Lemma 20) to treat every V_2-verified child as a single-segment leaf: *verification* of a proof for a statement outside a relation's support is not excluded by knowledge soundness, so without the typing the extractor could meet a verifying `convert` proof at a step count it cannot use. Completeness is unaffected — an honest prover's children have exactly these types. (The bounds $N_L, N_R \geq N_{\text{seg}}$ of the untyped formulation are implied by the typing.)

4.6 `embed` circuit

The `embed` circuit produces the final proof, asserting correctness of the entire program execution with public initial and final states. It assumes that the total step count T is known in advance to the verifier. We fix T as a publicly known system parameter satisfying $T \geq 2N_{\text{seg}}$ and $N_{\text{seg}} \mid T$ throughout, with $T \leq \mathsf{poly}(\lambda)$; these are standing assumptions on the system parameters and are not constraints enforced by the circuit itself. The lower bound $T \geq 2N_{\text{seg}}$ ensures that every execution has at least two segments and hence admits a well-formed `combine` proof tree: a single-segment execution ($T = N_{\text{seg}}$) cannot be proven, because $\mathcal{R}_3$ requires $N_L + N_R = N$ with $N_L, N_R \geq N_{\text{seg}}$, i.e. $N \geq 2N_{\text{seg}}$. Let $\Pi_3 = (\Pi_3.\mathsf{Prove}, \Pi_3.\mathsf{Verify})$ be the SNARK proving protocol for relation $\mathcal{R}_3$.

$$\mathcal{R}_4 = \left\{ \ ((\hat{S}_0,\, \hat{S}_T);\ \pi_3) \;\middle|\; \Pi_3.\mathsf{Verify}((\hat{S}_0,\, \hat{S}_T,\, T),\, \pi_3) = 1 \ \right\}$$

where $\hat{S}_0$ and $\hat{S}_T$ are the initial and final states of the entire program execution.

Remark 1 (Divisibility requirement on T)**.** *Since $\mathcal{R}_3$ requires both child step counts to be divisible by N_{seg}, every well-formed* `combine` *proof tree has a total step count N that is a multiple of N_{seg}. Consequently, the total step count T passed to the* `embed` *circuit must satisfy $N_{\text{seg}} \mid T$. The verifier checks this divisibility before invoking $\Pi_3.\mathsf{Verify}$; proofs for step counts not divisible by N_{seg} are rejected outright.*

Teams should explain how the relations are proven using AIR / IOPs / poly-IOPs and the BCS transform. That is, teams should provide a direct reduction from proving a relation to a low-degree test with FRI or another protocol. The reductions should contain links to the circuits submitted to soundcalc. The purpose of the section is to make explicit how the relations from the previous section are proven.

5 Security

We prove that the zkVM construction is *correct-trace extractable*: any adversary that produces a convincing final proof can be used to extract an explicit, valid execution trace. The proof proceeds by descending the recursion tree layer by layer, reducing everything ultimately to the knowledge soundness of the inner-circuit SNARKs, the binding properties of the memory commitment scheme $\mathrm{Com}_{\mathsf{mem}}$, and the collision-resistance of $\mathrm{Com}_{\mathsf{bus}}$.

Remark 2 (Idealized proof model)**.** *Our proof is idealized/heuristic in two respects.*

First, Definition 3 below postulates a straight-line extractor that runs in essentially the time of the adversary, with only additive $\mathsf{poly}(\lambda)$ overhead. Because our SNARKs are hash-based, the natural model is the random oracle model (ROM), in which such straight-line extractors do exist. The subtlety lies in the recursion: composing straight-line extraction across the layers of the proof tree (as in proof-carrying data, PCD) requires each SNARK to remain knowledge-sound relative to the same random oracle*, i.e. to be a* relativized *argument system, and such relativized SNARKs provably do not exist in general [1]. Our analysis idealizes over this obstruction: we assume that straight-line extraction composes across the recursion, effectively putting the non-existence of relativized SNARKs under the rug.*

Second, while the analysis does produce explicit advantage bounds and reduction running times (Theorem 27), the idealization above means these should not be read as a concrete security level: they are meaningful as a statement about the reduction structure — the recursion topology is sound and the circuit relations compose consistently — but a concrete security claim would require replacing the idealized straight-line extraction assumption with an analysis of the actual instantiation.

5.1 Definitions

We work with knowledge-sound argument systems admitting straight-line extraction. Let λ denote the security parameter; "PPT" means probabilistic polynomial time in λ.

Definition 3 (Knowledge-soundness)**.** *A non-interactive argument system $\Pi = (\mathsf{Prove}, \mathsf{Verify})$ for relation $\mathcal{R}$ is* knowledge-sound *if there exists a PPT algorithm $\mathcal{E}$ (the* extractor*) such that for every PPT adversary $\mathcal{A}$:*

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A}) \;:=\; \Pr\Big[(x,\pi) \leftarrow \mathcal{A}(1^\lambda);\; w \leftarrow \mathcal{E}(x,\pi) \;:\; \Pi.\mathsf{Verify}(x,\pi) = 1 \;\wedge\; (x;w) \notin \mathcal{R}\Big] \le \mathsf{negl}(\lambda). \tag{21}$$

We assume that the running time of $\mathcal{E}(x,\pi)$ is at most $\mathsf{Time}(\mathcal{A}(1^\lambda)) + \mathsf{poly}(\lambda)$ for every adversary $\mathcal{A}$ and every (x,π) in the support of $\mathcal{A}$'s output, i.e. the extractor incurs only an additive polynomial overhead over a single evaluation of $\mathcal{A}$ on the given proof. We call $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}(\mathcal{A})$ the knowledge-soundness error of $\mathcal{A}$ against Π.

Remark 4 (Straight-line extraction)**.** *The extractor $\mathcal{E}$ above is straight-line: it takes only the statement x and the proof π as input and does not rewind $\mathcal{A}$. This can actually not be achieved if π is succinct, but this is part of our (over-)idealization, see Remark 2 above. Namely, Definition 3 is an idealized requirement that cannot be satisfied by any standard construction in the plain model (see Remark 2). It serves as a* proxy*: we state it formally so that the reduction structure is explicit, with the understanding that concrete instantiations will replace it with ROM-based extractability at the cost of the overhead discussed in Remark 2.*

In the following, we omit commitment keys for simplicity of exposition. Teams should add appropriate definitions and adaptations if they rely on commitment keys.

We also leave out completeness definitions, as the document should just illustrate the structure of a potential soundness proof. Teams should add completeness arguments as well.

Definition 5 (Position-binding and update-binding vector commitment)**.** *A vector commitment scheme* $\mathrm{Com} = (\mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$ *satisfies the two independent properties below —* position-binding *and* update-binding *— which a scheme may have separately or together.*

1. ***Position-binding.*** *For every PPT adversary $\mathcal{A}$, the following is negligible:*

$$\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}}(\mathcal{A}) \;:=\; \Pr\left[\begin{array}{l}(C, i, v, \pi, v', \pi') \leftarrow \mathcal{A}(1^\lambda) \;: \\ \mathsf{Verify}(C, i, v, \pi) = 1 \;\wedge\; \mathsf{Verify}(C, i, v', \pi') = 1 \;\wedge\; v \neq v'\end{array}\right].$$

2. ***Update binding.*** *For every PPT adversary $\mathcal{A}$, the following is negligible, where $C := \mathsf{Commit}(\vec{m})$:*

$$\mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}}(\mathcal{A}) \;:=\; \Pr\left[\begin{array}{l}(\vec{m}, \mathtt{addr}, x, C', \pi) \leftarrow \mathcal{A}(1^\lambda) \;: \\ \mathsf{Verify}(C, \mathtt{addr}, \vec{m}[\mathtt{addr}], \pi) = 1 \;\wedge\; \mathsf{Verify}(C', \mathtt{addr}, x, \pi) = 1 \\ \wedge\; C' \neq \mathsf{Commit}(\vec{m}[\mathtt{addr} \mapsto x])\end{array}\right].$$

   In other words, a single opening proof π that opens an honest commitment $C = \mathsf{Commit}(\vec{m})$ at position `addr` *and also opens C' at* `addr` *to x forces C' to be the honest commitment of the updated vector $\vec{m}[\mathtt{addr} \mapsto x]$. In particular C' is then itself a well-formed commitment — an actual output of* Commit *— and not merely a value that verifies against some proofs.*

Remark 6 (Merkle trees and update binding)**.** *Standard Merkle tree commitments satisfy both position-binding and update binding under collision-resistance of the underlying hash function. The position-binding property is immediate. For update binding, a Merkle authentication path π for position* `addr` *consists of the sibling hashes along the root-to-leaf path; since $C = \mathsf{Commit}(\vec{m})$ is honest, those siblings must — except for a hash collision — be the honest siblings of $\vec{m}$, so recomputing the root with the new leaf x yields exactly $\mathsf{Commit}(\vec{m}[\mathtt{addr} \mapsto x])$, and no other value of C' can pass. We leave out a formal proof for now.*

Definition 7 (Collision-resistant bus commitment)**.** *A bus commitment scheme* $\mathrm{Com}_{\mathsf{bus}}$ *is* collision-resistant *if for every PPT adversary $\mathcal{A}$, the following is negligible:*

$$\mathsf{Adv}^{\mathsf{cr}}_{\mathrm{Com}_{\mathsf{bus}}}(\mathcal{A}) := \Pr\left[\begin{array}{l}(\widehat{\mathcal{B}}, \widehat{\mathcal{B}}') \leftarrow \mathcal{A}(1^\lambda) \;: \\ \widehat{\mathcal{B}} \neq \widehat{\mathcal{B}}' \;\wedge\; \mathrm{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) = \mathrm{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}')\end{array}\right].$$

Definition 8 (zkVM system)**.** *A* zkVM system *is a tuple*

$$\mathsf{zkVM} = \big(\{\varphi_{\mathrm{op}}\}_{\mathrm{op}},\ \mathrm{Com}_{\mathsf{bus}},\ \mathrm{Com}_{\mathsf{mem}},\ \Pi_0,\ \Pi_1,\ \Pi_2,\ \Pi_3,\ \Pi_4\big)$$

consisting of the following components.

1. ***Step predicates.*** *For each operation* op *(including* step*,* read*,* write*,* keccak*,* poseidon*,* range*), a predicate φ_{op} over VM states, expressible as a system of polynomial constraints over the field (as defined in Section 4).*

2. ***Bus commitment scheme.*** $\mathrm{Com}_{\mathsf{bus}}$ *is a collision-resistant bus commitment scheme (Definition 7) used to commit to the shared bus $\mathcal{B}$ in the inner-circuit relations $\mathcal{R}_{0,j}$.*

3. ***Memory commitment scheme.*** $\mathrm{Com}_{\mathsf{mem}} = (\mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$*, a position-binding and update-binding vector commitment scheme (Definition 5) used to represent the memory component of each committed VM state $\hat{S}$.*

4. ***SNARK hierarchy.***

   - $\Pi_0 = \{\Pi_{0,j}\}_{j\in\{\mathrm{step,keccak,poseidon,range}\}}$*: the four* inner-circuit *SNARKs for relations*

     $$\mathcal{R}_{0,\mathrm{step}},\ \mathcal{R}_{0,\mathrm{keccak}},\ \mathcal{R}_{0,\mathrm{poseidon}},\ \mathcal{R}_{0,\mathrm{range}},$$

     respectively.
   - Π_1*: the* `segment` *SNARK for relation $\mathcal{R}_1$ (verifies the four inner proofs).*
   - Π_2*: the* `convert` *SNARK for relation $\mathcal{R}_2$ (1-to-1 normalization).*
   - Π_3*: the* `combine` *SNARK for relation $\mathcal{R}_3$ (2-to-1 recursive merging).*
   - Π_4*: the* `embed` *SNARK for relation $\mathcal{R}_4$ (final proof).*

   Each $\Pi_i = (\Pi_i.\mathsf{Prove}, \Pi_i.\mathsf{Verify})$ is a non-interactive argument system for the respective relation.

Definition 9 (Correct-execution relation)**.** *Fix a program code* `code` *and a step count T satisfying $N_{\mathrm{seg}} \mid T$, $T \geq 2N_{\mathrm{seg}}$, and $T \leq \mathsf{poly}(\lambda)$ (system parameters, as in the* `embed` *circuit of Section 4); neither* `code` *nor T is chosen by the adversary.*

The correct-execution relation $\mathcal{R}^* = \mathcal{R}^*_{\mathtt{code},T}$ *has as statements boundary pairs (S_0, S_T) of full-memory VM states $S = (\mathtt{pc}, \mathtt{regs}, \mathtt{mem})$, and as witnesses candidate full-memory traces $(S'_0, \ldots, S'_T)$:*

$$\mathcal{R}^* \ =\ \Big\{\ \big((S_0, S_T);\ (S'_0, \ldots, S'_T)\big)\ \Big|\ S'_0 = S_0\ \wedge\ S'_T = S_T\ \wedge\ \forall i \in [T] : \varphi_{\mathrm{step}}(S'_{i-1}, S'_i) = \mathrm{TRUE}\Big\}, \tag{22}$$

where φ_{step} is the step predicate of Section 4 (the disjunction over all operation predicates φ_{op}).

Definition 10 (Final argument system)**.** *The* final argument system *of* zkVM *is the non-interactive argument system $\Pi^* = (\Pi^*.\mathsf{Prove}, \Pi^*.\mathsf{Verify})$ for the correct-execution relation $\mathcal{R}^*$ (Definition 9) whose proofs are the final* `embed` *proofs π^4 and whose verification algorithm, on statement (S_0, S_T) and proof π^4, proceeds as follows:*

1. *set*

   $$\hat{S}_0 := (\mathtt{pc}_0, \mathtt{regs}_0, \mathsf{Commit}(\mathtt{mem}_0)), \quad \hat{S}_T := (\mathtt{pc}_T, \mathtt{regs}_T, \mathsf{Commit}(\mathtt{mem}_T));$$

2. *output $\Pi_4.\mathsf{Verify}((\hat{S}_0, \hat{S}_T), \pi^4)$,*

where Commit *is that of* $\mathrm{Com}_{\mathsf{mem}}$*. Its prover* $\Pi^*.\mathsf{Prove}$ *is the honest proving pipeline of Section 4 instantiated at* $(\mathtt{code}, T)$*; it plays no role in what follows.*[6]

Definition 11 (Correct-trace extractability)**.** *The zkVM* zkVM *is* correct-trace extractable *if its final argument system* Π^* *(Definition 10) is knowledge-sound for the correct-execution relation* $\mathcal{R}^*$ *(Definition 9) in the sense of Definition 3. We write*

$$\mathsf{Adv}^{\mathsf{cte}}_{\mathsf{zkVM}}(\mathcal{A}) \;:=\; \mathsf{Adv}^{\mathsf{ks}}_{\Pi^*}(\mathcal{A})$$

for the CTE advantage *of a PPT adversary* $\mathcal{A}$*, and* $\mathcal{E}_{\mathsf{zkVM}}$ *for the extractor whose existence is asserted; the additive running-time convention of Definition 3 applies to it unchanged.*

Remark 12 (Unrolling Definition 11)**.** *Written out, Definition 11 says that there is a PPT extractor* $\mathcal{E}_{\mathsf{zkVM}}$ *such that for every PPT adversary* $\mathcal{A}$ *the experiment*

$$\mathsf{Exp}^{\mathsf{cte}}_{\mathsf{zkVM},\mathcal{E}_{\mathsf{zkVM}},\mathcal{A}}(\lambda)$$

below returns 1 *with at most negligible probability.*

1. ***Adversary.*** $\mathcal{A}(1^\lambda)$ *outputs a statement* (S_0, S_T) — *an initial state* $S_0 = (\mathtt{pc}_0, \mathtt{regs}_0, \mathtt{mem}_0)$ *and a final state* $S_T = (\mathtt{pc}_T, \mathtt{regs}_T, \mathtt{mem}_T)$ — *together with a proof* π^4*; the program code* $\mathtt{code}$ *and step count* T *are fixed system parameters.*
2. ***Verification.*** *If* $\Pi^*.\mathsf{Verify}((S_0, S_T), \pi^4) = 0$ *(Definition 10), return* 0*.*
3. ***Extraction.*** *Run* $\mathcal{E}_{\mathsf{zkVM}}(S_0, S_T, \pi^4)$ *to obtain full-memory VM states* $S'_0, S'_1, \ldots, S'_T$*, where* $S'_i = (\mathtt{pc}'_i, \mathtt{regs}'_i, \mathtt{mem}'_i)$*.*
4. ***Winning condition.*** *Return* 1 *(adversary wins) if and only if* $\big((S_0, S_T); (S'_0, \ldots, S'_T)\big) \notin \mathcal{R}^*$*, i.e. if any of the following hold:*
   - $\mathtt{pc}'_0 \neq \mathtt{pc}_0$*,* $\mathtt{regs}'_0 \neq \mathtt{regs}_0$*, or* $\mathtt{mem}'_0 \neq \mathtt{mem}_0$*; or*
   - $\mathtt{pc}'_T \neq \mathtt{pc}_T$*,* $\mathtt{regs}'_T \neq \mathtt{regs}_T$*, or* $\mathtt{mem}'_T \neq \mathtt{mem}_T$*; or*
   - $\varphi_{\mathrm{step}}(S'_i, S'_{i+1}) \neq \mathrm{TRUE}$ *for some* $i \in \{0, \ldots, T-1\}$*.*

This is Definition 3 instantiated at $(\mathcal{R}^*, \Pi^*)$ *verbatim; in particular the boundary checks are not extra requirements bolted onto the game, but precisely the two boundary conjuncts of* (22)*. Note also that a statement of* $\mathcal{R}^*$ *carries the full memory vectors* $\mathtt{mem}_0, \mathtt{mem}_T$*, so* $\Pi^*.\mathsf{Verify}$ *is not succinct; succinctness is a property of* Π_4 *and is orthogonal to correct-trace extractability.*

5.2 Memory extractability

The inner circuits operate on committed-memory states $\hat{S}$ and a bus $\widehat{\mathcal{B}}$, whereas the correct-execution relation $\mathcal{R}^*$ (22) requires a valid execution trace in terms of full-memory states S with *explicitly reconstructed* memory vectors $\mathtt{mem}_k$. The following proposition bridges this gap and shows that, once the commitment openings are embedded in the SNARK witnesses, the bus-based predicate implies the bus-free predicate: introducing a bus does not weaken the correctness guarantee. In particular, position-binding and update-binding of $\mathrm{Com}_{\mathsf{mem}}$ bound the probability that

[6] Observe that the definition of knowledge soundness just depends on a verification algorithm and not on the associated prover.

the extracted full-memory trace deviates from the committed-memory execution recorded by the SNARKs.

It does so in three respects: (i) it eliminates the bus; (ii) it lifts the step predicate from committed-memory states to full-memory states whose values at accessed addresses are uniquely determined by the position-binding commitment openings embedded in the SNARK witness; and (iii) it carries the commitment invariant $\hat{S}.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem})$ forward by one step, so that the invariant is a *conclusion* of the proposition rather than a side condition established elsewhere. Concretely, it shows that if the bus-parameterised step predicate $\widehat{\varphi}_{\mathrm{step}}$ holds (as guaranteed by Lemma 24), then the bus-free, full-memory predicate $\varphi_{\mathrm{step}}(S_1, S_2)$ holds for the states $S_j := (\hat{S}_j.\mathtt{pc}, \hat{S}_j.\mathtt{regs}, \mathtt{mem}_j)$ formed with the reconstructed memory vectors, and $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$; the required memory values are extracted from the commitment openings embedded in the definitions of $\widehat{\varphi}_{\mathrm{read}}$ and $\widehat{\varphi}_{\mathrm{write}}$. It is invoked once per step in Step 6 of the proof of Theorem 27, where the two conclusions are used respectively as the per-step guarantee required by the step conjunct of (22) and as the induction hypothesis for the next step.

The second memory vector is not a free choice of the adversary: it is obtained from the first by the deterministic *memory reconstruction rule*

$$\mathtt{mem}_2 \;:=\; \begin{cases} \mathtt{mem}_1[\,\hat{S}_1.\mathtt{regs}[0] \mapsto \hat{S}_1.\mathtt{regs}[1]\,] & \text{if } \mathtt{code}[\hat{S}_1.\mathtt{pc}] = \text{write}, \\ \mathtt{mem}_1 & \text{otherwise}, \end{cases} \tag{23}$$

which is exactly the rule the extractor of Theorem 27 applies when it reconstructs the full-memory trace. Fixing $\mathtt{mem}_2$ this way is what makes the commitment invariant provable rather than assumable: an adversary that could choose $\mathtt{mem}_2$ freely subject to $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$ would already have been handed the invariant as a hypothesis, and the update-binding term in the bound below would do no work.

Throughout, we write

$$\widehat{\varphi}_{\mathrm{step}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}})$$

as shorthand for the four-argument predicate (14) instantiated with the memory-opening witness π_i^{mem} that is carried alongside the states:

$$\widehat{\varphi}_{\mathrm{step}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi_i^{\mathsf{mem}}) = \mathrm{TRUE}.$$

The shorthand is *not* an existential quantification over π_i^{mem}: the witness is a fixed entry of the extracted $\mathcal{R}_{0,\mathrm{step}}$ witness vector. Wherever the shorthand and an explicit memory-opening witness both occur — as in Proposition 13, whose second and fourth hypotheses each refer to π^{mem} — they denote the same object, so the disjunct of $\widehat{\varphi}_{\mathrm{step,bus}}$ that is satisfied is the one evaluated on that witness.

Proposition 13 (Memory extractability)**.** *For every PPT adversary $\mathcal{A}$ that outputs a tuple*

$$(\hat{S}_1, \hat{S}_2, \mathtt{mem}_1, \widehat{\mathcal{B}}, \pi^{\mathsf{mem}})$$

such that, defining $\mathtt{mem}_2$ *by the reconstruction rule* (23) *and writing* $S_j := (\hat{S}_j.\mathtt{pc}, \hat{S}_j.\mathtt{regs}, \mathtt{mem}_j)$*, the following hold with probability ε:*

- $\hat{S}_1.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_1)$*;*
- $\widehat{\varphi}_{\mathrm{step}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) = \mathrm{TRUE}$*;*

- *the step fails to transfer, i.e. the commitment invariant is not carried forward or the full-memory step predicate does not hold:*

$$\hat{S}_2.\mathtt{mem} \neq \mathsf{Commit}(\mathtt{mem}_2) \qquad \textit{or} \qquad \varphi_{\mathrm{step}}(S_1, S_2) \neq \mathrm{TRUE};$$

- *the memory-opening witness* π^{mem} *satisfies, depending on the operation type:*
  - Read*:* $\pi^{\mathsf{mem}} = \pi$ *with* $\hat{S}_1.\mathtt{mem} = \hat{S}_2.\mathtt{mem}$ *and* $\mathsf{Verify}(\hat{S}_1.\mathtt{mem}, \hat{S}_1.\mathtt{regs}[0], \hat{S}_2.\mathtt{regs}[1], \pi) = 1$*;*
  - Write*:* $\pi^{\mathsf{mem}} = (\pi_{\mathrm{write}}, v_{\mathsf{old}})$ *with*

$$\mathsf{Verify}(\hat{S}_1.\mathtt{mem}, \hat{S}_1.\mathtt{regs}[0], v_{\mathsf{old}}, \pi_{\mathrm{write}}) = 1,$$
$$\mathsf{Verify}(\hat{S}_2.\mathtt{mem}, \hat{S}_1.\mathtt{regs}[0], \hat{S}_1.\mathtt{regs}[1], \pi_{\mathrm{write}}) = 1;$$

  - Other*: no condition is imposed on* π^{mem}*;*

there exist PPT reduction adversaries $\mathcal{D}^{\mathsf{pos}}$ *and* $\mathcal{D}^{\mathsf{upd}}$*, each constructed from* $\mathcal{A}$ *and running in time* $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$*, such that*

$$\varepsilon \leq \ \mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}) + \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}}).$$

Equivalently: whenever the first, second and fourth hypotheses hold, both $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$ *and* $\varphi_{\mathrm{step}}(S_1, S_2) = \mathrm{TRUE}$*, except with probability at most* $\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}) + \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}})$*.*

Remark 14 (On the fourth hypothesis)**.** *Two points about the case distinction in the fourth hypothesis.*

First, the Other *case is deliberately unconstrained. The remaining disjuncts of* (14) *never read* π^{mem}*, so nothing in the construction forces its value on a non-memory step, and the extractor of Lemma 24 returns whatever the prover placed in that witness field. Demanding* $\pi^{\mathsf{mem}} = \perp$ *there would be a requirement the zkVM does not enforce, and would make the proposition inapplicable to exactly those steps; the proof never uses it.*

Second, which of the three cases applies is determined by $\mathrm{op} := \mathtt{code}[\hat{S}_1.\mathtt{pc}]$ *and is therefore not a free choice of* $\mathcal{A}$*. Every* φ'_{op} *carries the fetch conjunct* $\mathtt{code}[\mathtt{pc}_1] = \mathrm{op}$ *of* (5) *— for the two precompiles it is carried by the chip predicate* $\widehat{\varphi}_{\mathrm{op}}(\widehat{\mathcal{B}})$ *rather than by the bus-membership disjunct itself — so under the second hypothesis at most one disjunct of* $\widehat{\varphi}_{\mathrm{step,bus}}$ *can be satisfied. The operation type is thus well defined, and the case split in the proof below is the same case split as the one above.*

Proof. We define two bad events, each a violation of one binding property of $\mathrm{Com}_{\mathsf{mem}}$, and bound each by a single reduction that runs $\mathcal{A}$ once. We then show (Step A) that off both events the assumption $\widehat{\varphi}_{\mathrm{step}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) = \mathrm{TRUE}$ forces both $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$ and $\varphi_{\mathrm{op}}(S_1, S_2) = \mathrm{TRUE}$ for the operation op selected by the satisfied disjunct, and hence $\varphi_{\mathrm{step}}(S_1, S_2) = \mathrm{TRUE}$ — contradicting the third winning condition. A union bound then gives the claim.

Bad events. Run $\mathcal{A}(1^\lambda)$ to obtain $(\hat{S}_1, \hat{S}_2, \mathtt{mem}_1, \widehat{\mathcal{B}}, \pi^{\mathsf{mem}})$ and set $\mathtt{mem}_2$ by (23). By the first hypothesis $\hat{S}_1.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_1)$, correctness of $\mathrm{Com}_{\mathsf{mem}}$ makes the honest opening $\mathsf{Open}(\mathtt{mem}_1, p)$ verify against $\hat{S}_1.\mathtt{mem}$ at every position p.

All memory vectors have the same length n, fixed by the system parameters, and positions range over the index domain $[n]$. Writing $\mathtt{addr} := \hat{S}_1.\mathtt{regs}[0]$ and $x := \hat{S}_1.\mathtt{regs}[1]$, let

$$\mathcal{V} \ := \ \{\, \mathtt{mem}_1,\ \mathtt{mem}_2 \,\}$$

be the *candidate vectors*. Both have length n and are computable in time $\mathsf{poly}(\lambda)$ from $\mathcal{A}$'s output — $\mathtt{mem}_2$ because the reconstruction rule (23) is deterministic and explicit — so a reduction may open either of them honestly at any position. Note that on a write step $\mathtt{mem}_2$ *is* the updated vector $\mathtt{mem}_1[\mathtt{addr} \mapsto x]$, so $\mathcal{V}$ already contains every vector the argument below opens. The well-formedness conjunct of φ'_{read} and φ'_{write} guarantees $\mathtt{addr} \in [n]$ on the two memory operations, so $\mathtt{mem}_1[\mathtt{addr}]$, $\mathsf{Open}(\mathtt{mem}_1, \mathtt{addr})$ and $\mathtt{mem}_1[\mathtt{addr} \mapsto x]$ are all well defined there. Define:

- E_{pos} (*position-binding collision*): there are a root $R \in \{\hat{S}_1.\mathtt{mem}, \hat{S}_2.\mathtt{mem}\}$, a position p, and two accepting openings of R at p to distinct values, each either read from π^{mem} or computed as an honest opening $\mathsf{Open}(\vec{v}, p)$ of a candidate vector $\vec{v} \in \mathcal{V}$.
- E_{upd} (*update-binding collision*): $\pi^{\mathsf{mem}} = (\pi_{\text{write}}, v_{\mathsf{old}})$ is a write witness whose shared path π_{write} opens the honest commitment $\hat{S}_1.\mathtt{mem}$ at $\mathtt{addr} := \hat{S}_1.\mathtt{regs}[0]$ to $\mathtt{mem}_1[\mathtt{addr}]$ and opens $\hat{S}_2.\mathtt{mem}$ at $\mathtt{addr}$ to $x := \hat{S}_1.\mathtt{regs}[1]$, yet $\hat{S}_2.\mathtt{mem} \neq \mathsf{Commit}(\mathtt{mem}_1[\mathtt{addr} \mapsto x])$.

The two reductions. $\mathcal{D}^{\mathsf{pos}}$ runs $\mathcal{A}(1^\lambda)$ once, then compares the candidate vectors in $\mathcal{V}$ pairwise and against the values opened by π^{mem}; whenever E_{pos} occurs it computes the corresponding honest openings and outputs the two openings (R, p, v, ρ) and (R, p, v', ρ') with $v \neq v'$ to the position-binding challenger, breaking it. Hence $\Pr[E_{\mathsf{pos}}] \leq \mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}})$. $\mathcal{D}^{\mathsf{upd}}$ runs $\mathcal{A}(1^\lambda)$ once; whenever E_{upd} occurs it outputs

$$(\mathtt{mem}_1, \mathtt{addr}, x, \hat{S}_2.\mathtt{mem}, \pi_{\text{write}})$$

to the update-binding challenger, breaking it: $\hat{S}_1.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_1)$ is honest, the shared path π_{write} opens it at $\mathtt{addr}$ to $\mathtt{mem}_1[\mathtt{addr}]$ and opens $\hat{S}_2.\mathtt{mem}$ at $\mathtt{addr}$ to x, yet $\hat{S}_2.\mathtt{mem} \neq \mathsf{Commit}(\mathtt{mem}_1[\mathtt{addr} \mapsto x])$. Hence $\Pr[E_{\mathsf{upd}}] \leq \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}})$. Each reduction makes one run of $\mathcal{A}$, a linear scan of the candidate vectors and a constant number of honest openings, so both run in time $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ (note that $\mathcal{A}$ itself outputs $\mathtt{mem}_1$ and hence already runs in time $\Omega(n)$).

Step A: off both events, the invariant and the step predicate transfer. *Goal:* assuming $\neg E_{\mathsf{pos}} \wedge \neg E_{\mathsf{upd}}$,

$$\widehat{\varphi}_{\text{step}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) = \text{TRUE}$$

implies $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$ and $\varphi_{\text{step}}(S_1, S_2) = \text{TRUE}$. For the second conclusion we prove the sharper statement that $\varphi_{\text{op}}(S_1, S_2) = \text{TRUE}$ for the operation op whose disjunct of $\widehat{\varphi}_{\text{step,bus}}$ is satisfied; since $\varphi_{\text{step}}(S_1, S_2)$ is by definition the disjunction of the $\varphi_{\text{op}}(S_1, S_2)$ over all operations, the goal follows at once. Every statement below is deterministic given $\neg E_{\mathsf{pos}} \wedge \neg E_{\mathsf{upd}}$. By Eq. (14), the assumption expands as

$$\widehat{\varphi}_{\text{step}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) \;=\; \widehat{\varphi}_{\text{step,bus}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) \;\wedge\; \widehat{\varphi}_{\text{keccak}}(\widehat{\mathcal{B}}) \;\wedge\; \widehat{\varphi}_{\text{poseidon}}(\widehat{\mathcal{B}}) \;\wedge\; \widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) \;=\; \text{TRUE},$$

so in particular $\widehat{\varphi}_{\text{step,bus}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) = \text{TRUE}$ and all chip predicates hold.

(No full-memory bus.) The argument goes directly from the committed, bus-parameterised predicate to the bus-free, full-memory predicate $\varphi_{\text{step}}(S_1, S_2)$; it never constructs a full-memory bus $\mathcal{B}$ and never invokes the bus-parameterised form (10) of the step predicate. This is deliberate, and necessary: $\widehat{\mathcal{B}}$ is the bus of a whole segment (Lemma 24), so it carries range-check and precompile entries belonging to *other* steps, for which no memory vectors are extracted and no full-memory states exist. A predicate such as $\varphi_{\text{keccak}}(\mathcal{B})$, which quantifies over all entries of the bus, therefore

could not even be stated here, let alone established. Instead each bus-deferred obligation enters the case analysis on the committed side only, via the chip predicates $\widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}})$ and $\widehat{\varphi}_{\text{op}}(\widehat{\mathcal{B}})$ above, and is consumed at the single bus entry belonging to this step.

(No commitment-injectivity argument.) Because $\texttt{mem}_2$ is fixed by the reconstruction rule (23) rather than chosen by $\mathcal{A}$, the case analysis never has to infer an equality of vectors from an equality of their commitments: each case obtains the memory component of $\varphi_{\text{op}}(S_1, S_2)$ directly from (23), and obtains the invariant $\hat{S}_2.\texttt{mem} = \mathsf{Commit}(\texttt{mem}_2)$ either from the commitment-equality conjunct of the disjunct (all cases but write) or from update binding (the case write). Position binding is used only to pin the *value* at the accessed address, at the single position `addr`.

> Here, and in some other places of the proofs, we implicitly used correctness of the vector commitment scheme, which we never explicitly defined. Team are expected to be less sloppy in this regard.

We case-split on the satisfied disjunct of $\widehat{\varphi}_{\text{step,bus}}$ (cf. (14)). By Remark 14 the satisfied disjunct is the one belonging to $\text{op} = \texttt{code}[\hat{S}_1.\texttt{pc}]$, so the case distinction below is the same one that drives the reconstruction rule (23).

(Invariant, all cases but write*.)* In the four non-write cases $\text{op} \neq \text{write}$, so (23) sets $\texttt{mem}_2 := \texttt{mem}_1$, and the satisfied disjunct asserts the commitment equality $\hat{S}_1.\texttt{mem} = \hat{S}_2.\texttt{mem}$. The invariant is then immediate,

$$\hat{S}_2.\texttt{mem} = \hat{S}_1.\texttt{mem} = \mathsf{Commit}(\texttt{mem}_1) = \mathsf{Commit}(\texttt{mem}_2),$$

with no appeal to either binding property, and the memory conjunct $\texttt{mem}_2 = \texttt{mem}_1$ of φ_{op} holds by construction. We record this once and do not repeat it in the four cases below; only the write case, where the commitment genuinely changes, needs update binding.

- **Non-memory arithmetic** $(\text{op}_1, \ldots, \text{op}_{10})$. The disjunct asserts $\widehat{\varphi}_{\text{op}}(\hat{S}_1, \hat{S}_2) \wedge \hat{S}_1.\texttt{mem} = \hat{S}_2.\texttt{mem}$. The arithmetic part gives $\varphi'_{\text{op}}(\hat{S}_1.\texttt{pc}, \hat{S}_1.\texttt{regs}, \hat{S}_2.\texttt{pc}, \hat{S}_2.\texttt{regs}) = \text{TRUE}$, and $\texttt{mem}_2 = \texttt{mem}_1$ by (23). Therefore $\varphi_{\text{op}}(S_1, S_2) = \varphi'_{\text{op}}(S_1, S_2) \wedge (\texttt{mem}_2 = \texttt{mem}_1) = \text{TRUE}$.

- **Arithmetic with range checks** $(\text{op}_{11}, \ldots, \text{op}_{20})$. The arithmetic part

  $$\widehat{\varphi}'_{\text{op}}(\hat{S}_1, \hat{S}_2)$$

  is memory-free, so $\varphi'_{\text{op}}(S_1, S_2) = \text{TRUE}$. The range-check requirement $(\text{range}, \hat{S}_1) \in \widehat{\mathcal{B}}$ and

  $$\widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) = \text{TRUE}$$

  together give $\varphi_{\text{range}}(S_1) = \text{TRUE}$ (range checks depend only on registers, not memory), and $\texttt{mem}_2 = \texttt{mem}_1$ by (23). Hence

  $$\varphi_{\text{op}}(S_1, S_2) = \varphi'_{\text{op}}(S_1, S_2) \wedge \varphi_{\text{range}}(S_1) \wedge (\texttt{mem}_2 = \texttt{mem}_1) = \text{TRUE}.$$

- **Read** (read). Here $\pi^{\mathsf{mem}} = \pi$ with $\hat{S}_1.\texttt{mem} = \hat{S}_2.\texttt{mem}$ and

  $$\mathsf{Verify}(\hat{S}_1.\texttt{mem}, \hat{S}_1.\texttt{regs}[0], \hat{S}_2.\texttt{regs}[1], \pi) = 1.$$

  Write $\texttt{addr} := \hat{S}_1.\texttt{regs}[0]$ and $x := \hat{S}_2.\texttt{regs}[1]$. The honest opening $\mathsf{Open}(\texttt{mem}_1, \texttt{addr})$ verifies against $\hat{S}_1.\texttt{mem}$ to $\texttt{mem}_1[\texttt{addr}]$; were $\texttt{mem}_1[\texttt{addr}] \neq x$, this and π would witness E_{pos} at $(R, p) = (\hat{S}_1.\texttt{mem}, \texttt{addr})$, which is excluded. So $\texttt{mem}_1[\texttt{addr}] = x$, which is the memory-access conjunct

$\mathtt{mem}_1[\mathtt{regs}_1[0]] = \mathtt{regs}_2[1]$ of (8). The memory-preservation conjunct $\mathtt{mem}_2 = \mathtt{mem}_1$ of (8) holds by (23). The arithmetic part satisfies

$$\varphi'_{\mathrm{read}}(\hat{S}_1.\mathtt{pc}, \hat{S}_1.\mathtt{regs}, \hat{S}_2.\mathtt{pc}, \hat{S}_2.\mathtt{regs}) = \mathrm{TRUE}$$

by Eq. (12). All three conjuncts hold, so $\varphi_{\mathrm{read}}(S_1, S_2) = \mathrm{TRUE}$ per Eq. (8).

- **Write** (write). Here $\pi^{\mathsf{mem}} = (\pi_{\mathrm{write}}, v_{\mathsf{old}})$ with

$$\mathsf{Verify}(\hat{S}_1.\mathtt{mem}, \hat{S}_1.\mathtt{regs}[0], v_{\mathsf{old}}, \pi_{\mathrm{write}}) = 1,$$
$$\mathsf{Verify}(\hat{S}_2.\mathtt{mem}, \hat{S}_1.\mathtt{regs}[0], \hat{S}_1.\mathtt{regs}[1], \pi_{\mathrm{write}}) = 1.$$

  Write $\mathtt{addr} := \hat{S}_1.\mathtt{regs}[0]$ and $x := \hat{S}_1.\mathtt{regs}[1]$, so that $\mathtt{mem}_2 = \mathtt{mem}_1[\mathtt{addr} \mapsto x]$ by (23). The honest opening of $\mathtt{mem}_1$ at $\mathtt{addr}$ verifies against $\hat{S}_1.\mathtt{mem}$ to $\mathtt{mem}_1[\mathtt{addr}]$; were $\mathtt{mem}_1[\mathtt{addr}] \neq v_{\mathsf{old}}$, this and π_{write} would witness E_{pos} under $\hat{S}_1.\mathtt{mem}$. So off E_{pos}, $\mathtt{mem}_1[\mathtt{addr}] = v_{\mathsf{old}}$; that is, the shared path π_{write} opens the honest commitment $\hat{S}_1.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_1)$ at $\mathtt{addr}$ to $\mathtt{mem}_1[\mathtt{addr}]$ and opens $\hat{S}_2.\mathtt{mem}$ at $\mathtt{addr}$ to x. Off E_{upd} this forces

$$\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_1[\mathtt{addr} \mapsto x]) = \mathsf{Commit}(\mathtt{mem}_2),$$

  which is exactly the invariant for this case — the one place in the proof where update binding is used, and the reason $\mathrm{Com}_{\mathsf{mem}}$ is required to have that property at all. The memory conjunct $\mathtt{mem}_2 = \mathtt{mem}_1[\mathtt{regs}_1[0] \mapsto \mathtt{regs}_1[1]]$ of (9) holds by (23), and with the arithmetic part

$$\varphi'_{\mathrm{write}}(\hat{S}_1.\mathtt{pc}, \hat{S}_1.\mathtt{regs}, \hat{S}_2.\mathtt{pc}, \hat{S}_2.\mathtt{regs}) = \mathrm{TRUE}$$

  from Eq. (13) we get $\varphi_{\mathrm{write}}(S_1, S_2) = \mathrm{TRUE}$ per Eq. (9).

- **Keccak/Poseidon precompile.** The disjunct of $\widehat{\varphi}_{\mathrm{step,bus}}$ asserts $(\mathrm{op}, \hat{S}_1, \hat{S}_2) \in \widehat{\mathcal{B}}$. Since $\widehat{\varphi}_{\mathrm{op}}(\widehat{\mathcal{B}}) = \mathrm{TRUE}$, the entry $(\mathrm{op}, \hat{S}_1, \hat{S}_2)$ satisfies $\widehat{\varphi}_{\mathrm{op}}(\hat{S}_1, \hat{S}_2) = \mathrm{TRUE}$, where φ_{op} (for $\mathrm{op} \in \{\mathrm{keccak}, \mathrm{poseidon}\}$) is the predicate defined in Section 4 ("Predicates for individual operations") asserting correctness of the precompile call. Like every non-memory operation, φ_{op} decomposes as $\varphi'_{\mathrm{op}}(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{pc}_2, \mathtt{regs}_2) \wedge (\mathtt{mem}_2 = \mathtt{mem}_1)$, and its pc-and-register part φ'_{op} — the part that actually asserts correctness of the precompile call — is memory-free and hence literally identical to $\widehat{\varphi}'_{\mathrm{op}}$. We therefore obtain $\varphi'_{\mathrm{op}}(S_1, S_2) = \mathrm{TRUE}$. The remaining conjunct is $\mathtt{mem}_2 = \mathtt{mem}_1$, which holds by (23), as expected for a precompile that does not modify main memory. Hence $\varphi_{\mathrm{op}}(S_1, S_2) = \varphi'_{\mathrm{op}}(S_1, S_2) \wedge (\mathtt{mem}_2 = \mathtt{mem}_1) = \mathrm{TRUE}$.

In every case the invariant $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$ holds, and the bus-free, full-memory predicate $\varphi_{\mathrm{op}}(S_1, S_2) = \mathrm{TRUE}$ holds for the operation op selected by the satisfied disjunct. Since $\varphi_{\mathrm{step}}(S_1, S_2)$ is the disjunction of the $\varphi_{\mathrm{op}}(S_1, S_2)$ over all operations, the latter gives $\varphi_{\mathrm{step}}(S_1, S_2) = \mathrm{TRUE}$, and both goals of Step A are established. Note that the bus and the commitments have now disappeared from the second conclusion, so no separate bus-elimination step is required.

Conclusion. Suppose $\mathcal{A}$ wins, i.e. all four hypotheses hold; in particular $\widehat{\varphi}_{\mathrm{step}}(\hat{S}_1, \hat{S}_2, \widehat{\mathcal{B}}) = \mathrm{TRUE}$ and at least one of $\hat{S}_2.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_2)$, $\varphi_{\mathrm{step}}(S_1, S_2) = \mathrm{TRUE}$ fails. If neither E_{pos} nor E_{upd} occurred, Step A would give both, a contradiction; hence at least one bad event occurs whenever $\mathcal{A}$ wins. By a union bound,

$$\varepsilon \;\leq\; \Pr[E_{\mathsf{pos}}] + \Pr[E_{\mathsf{upd}}] \;\leq\; \mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}) + \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}}).$$

□

Remark 15 (Memory reconstruction invariant)**.** *In Theorem 27 Step 6 the memory vectors* $\mathtt{mem}_k$ *are reconstructed inductively:* $\mathtt{mem}_0$ *is read off the initial state, and* $\mathtt{mem}_{k+1}$ *is obtained from* $\mathtt{mem}_k$ *by the reconstruction rule* (23) *applied to the step* $(\hat{S}_k, \hat{S}_{k+1})$*. The claim carried along the induction is the* commitment invariant

$$\hat{S}_k.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k) \qquad \textit{for all } k.$$

The invariant is not *an independent observation requiring its own appeal to the two binding properties: it is the first conclusion of Proposition 13. The base case* $k = 0$ *holds by construction, and the inductive step is one invocation of the proposition, which — from* $\hat{S}_k.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k)$ *and* $\widehat{\varphi}_{\mathrm{step}}(\hat{S}_k, \hat{S}_{k+1}, \widehat{\mathcal{B}}) = \mathrm{TRUE}$ *— returns both* $\hat{S}_{k+1}.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_{k+1})$*, which carries the induction, and* $\varphi_{\mathrm{step}}(S_k, S_{k+1}) = \mathrm{TRUE}$*, which is the per-step obligation of* (22)*. Both cost the same single pair of binding events, so the two guarantees are obtained together rather than paid for twice. Within the proposition the work splits as follows:*

- ***Write step*** *at address* $\mathtt{addr}$ *with new value* x*: the commitment genuinely changes, and only here is update binding used. Position binding first identifies* v_{old} *with* $\mathtt{mem}_k[\mathtt{addr}]$*, which turns* π_{write} *into an opening of the* honest *commitment* $\hat{S}_k.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k)$*; update binding then forces* $\hat{S}_{k+1}.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k[\mathtt{addr} \mapsto x]) = \mathsf{Commit}(\mathtt{mem}_{k+1})$*.*
- ***All other steps****:* $\mathtt{mem}_{k+1} = \mathtt{mem}_k$ *by* (23) *and* $\hat{S}_{k+1}.\mathtt{mem} = \hat{S}_k.\mathtt{mem}$ *(enforced by the memory-equality conjunct of* $\widehat{\varphi}_{op}$ *in* (14)*), so the invariant is inherited with no cryptographic assumption at all.*

Position binding is additionally what pins the value returned by a read: for a read step at index k*, Eq.* (12) *provides* π *with*

$$\mathsf{Verify}(\hat{S}_k.\mathtt{mem}, \hat{S}_k.\mathtt{regs}[0], \hat{S}_{k+1}.\mathtt{regs}[1], \pi) = 1,$$

and together with the invariant $\hat{S}_k.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k)$ *this gives* $\hat{S}_{k+1}.\mathtt{regs}[1] = \mathtt{mem}_k[\hat{S}_k.\mathtt{regs}[0]]$ *— the memory-access conjunct of* (8)*. This is the read case of Step A of the proposition, stated here in the notation of Theorem 27.*

5.3 Lemmas: descending the recursion tree

We state one lemma per layer of the recursion tree, proceeding from the outermost (`embed`) circuit inward to the inner-circuit layer. Each lemma composes the straight-line extractors guaranteed by Definition 3.

Advantage notation. We collect the advantage notation introduced with the definitions above: $\mathsf{Adv}^{\mathsf{ks}}_{\Pi}$ (knowledge-soundness error, Definition 3); $\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}}$ and $\mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}}$ (position-binding and update-binding advantages for the memory commitment scheme $\mathrm{Com}_{\mathsf{mem}}$, Definition 5); and $\mathsf{Adv}^{\mathsf{cr}}_{\mathrm{Com}_{\mathsf{bus}}}$ (collision-resistance advantage for the bus hash function, Definition 7). All advantages are functions of the security parameter λ. In each lemma below we exhibit explicit reduction adversaries $\mathcal{D}$ and state their running times. When a bound has the form $k \cdot \mathsf{Adv}(\mathcal{D})$, the reduction $\mathcal{D}$ denotes the maximum-advantage adversary among the k per-node (or per-step) reductions constructed in the proof; each such reduction runs in

$$\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda).$$

5.3.1 Embed layer: base of the composition

Definition 16 (Embed extraction game)**.** *For a zkVM system, an extractor $\mathcal{E}_4$, and a PPT adversary $\mathcal{A}$, define the game $\mathsf{Exp}^{\mathsf{embed}}_{\mathcal{E}_4,\mathcal{A}}(\lambda)$:*

1. ***Adversary.*** *$\mathcal{A}(1^\lambda)$ outputs a statement $(\hat{S}_0, \hat{S}_T)$ and a proof π^4; the step count T is a fixed system parameter.*
2. ***Filter.*** *If $\Pi_4.\mathsf{Verify}((\hat{S}_0, \hat{S}_T), \pi^4) = 0$, return 0.*
3. ***Extraction.*** *Run $\pi^3 \leftarrow \mathcal{E}_4(\hat{S}_0, \hat{S}_T, \pi^4)$.*
4. ***Winning condition.*** *Return 1 iff $\Pi_3.\mathsf{Verify}((\hat{S}_0, \hat{S}_T, T), \pi^3) = 0$, i.e. the extractor failed to produce a valid $\mathcal{R}_4$ witness despite an accepting `embed` proof.*

Write $\mathsf{Adv}^{\mathsf{embed}}_{\mathcal{E}_4}(\mathcal{A}) := \Pr[\mathsf{Exp}^{\mathsf{embed}}_{\mathcal{E}_4,\mathcal{A}}(\lambda) = 1]$, where the probability is over the random coins of $\mathcal{A}$ and of the extractor $\mathcal{E}_4$ (the two Verify steps are deterministic); as in Definition 3, any setup/CRS or random-oracle sampling is left implicit.

Lemma 17 (Embed extraction)**.** *Assume Π_4 is knowledge-sound (Definition 3). We construct an extractor $\mathcal{E}_4$ (given in the proof) such that for every PPT adversary $\mathcal{A}$ there is a reduction adversary $\mathcal{D}_4$ with*

$$\mathsf{Adv}^{\mathsf{embed}}_{\mathcal{E}_4}(\mathcal{A}) \;\le\; \mathsf{Adv}^{\mathsf{ks}}_{\Pi_4}(\mathcal{D}_4),$$

where $\mathsf{Time}(\mathcal{D}_4) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. This is the base layer of the extractor composition; the bound is a direct instantiation of the knowledge soundness of Π_4, stated as a game only for uniformity with Lemmas 20–24.

Proof. Let $\mathcal{E}_{\Pi_4}$ be the knowledge-soundness extractor for Π_4 guaranteed by Definition 3, and set $\mathcal{E}_4 := \mathcal{E}_{\Pi_4}$ (the $\mathcal{R}_4$ witness is exactly π^3). Define the reduction adversary $\mathcal{D}_4$, *playing the knowledge-soundness game of Π_4 for relation $\mathcal{R}_4$*, as follows:

1. Run $((\hat{S}_0, \hat{S}_T), \pi^4) \leftarrow \mathcal{A}(1^\lambda)$.
2. Output the statement $x := (\hat{S}_0, \hat{S}_T)$ and proof π^4 to its own challenger.

$\mathcal{D}_4$ makes a single invocation of $\mathcal{A}$ and no other computation, so $\mathsf{Time}(\mathcal{D}_4) = \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

The knowledge-soundness game runs $\mathcal{E}_{\Pi_4}(x, \pi^4)$ to obtain a candidate witness, which by construction is the value π^3 returned by $\mathcal{E}_4$. Comparing the two games: both accept the same (x, π^4) at the filter step, and $\mathsf{Exp}^{\mathsf{embed}}$ returns 1 exactly when π^4 verifies but $(x; \pi^3) \notin \mathcal{R}_4$ — precisely the event that $\mathcal{D}_4$ wins the knowledge-soundness game of Π_4. Hence $\mathsf{Adv}^{\mathsf{embed}}_{\mathcal{E}_4}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_4}(\mathcal{D}_4)$, giving the stated bound. □

5.3.2 Combine layer: unrolling the recursion tree

We establish well-foundedness of the recursion tree.

Remark 18 (Well-foundedness of `combine`)**.** *By convention, every `combine` statement $(\hat{S}_0^L, \hat{S}_N^R, N)$ carries its step count N as a public input. The relation $\mathcal{R}_3$ enforces $N_{\text{seg}} \mid N_L$, $N_{\text{seg}} \mid N_R$, $N_L \ge N_{\text{seg}}$, $N_R \ge N_{\text{seg}}$, and $N_L + N_R = N$, so each child's step count is strictly smaller than its parent's. Hence the recursion tree is finite and strong induction on N is well-founded.*

Throughout this lemma we assume $2N_{\text{seg}} \le N \le T$ and $T \le \mathsf{poly}(\lambda)$ (the step count is a system parameter, Definition 9), so $m = N/N_{\text{seg}}$ is polynomially bounded.

Definition 19 (Combine extraction game)**.** *For an extractor $\mathcal{E}_3$, a step count N with $N_{\text{seg}} \mid N$ (write $m := N/N_{\text{seg}}$), and PPT $\mathcal{A}$, define the game $\mathsf{Exp}^{\mathsf{comb}}_{N,\mathcal{E}_3,\mathcal{A}}(\lambda)$:*

1. ***Adversary.*** *$\mathcal{A}(1^\lambda)$ outputs $(\hat{S}_0, \hat{S}_N, N)$ and a proof π^3.*
2. ***Filter.*** *If $\Pi_3.\mathsf{Verify}((\hat{S}_0, \hat{S}_N, N), \pi^3) = 0$, return 0.*
3. ***Extraction.*** *Run*

$$(\hat{S}^{(0)}, \ldots, \hat{S}^{(m)}; \pi^2_1, \ldots, \pi^2_m) \leftarrow \mathcal{E}_3(\hat{S}_0, \hat{S}_N, N, \pi^3).$$

4. ***Winning condition.*** *Return 1 unless $\hat{S}^{(0)} = \hat{S}_0$, $\hat{S}^{(m)} = \hat{S}_N$, and*

$$\Pi_2.\mathsf{Verify}((\hat{S}^{(i-1)}, \hat{S}^{(i)}, N_{\text{seg}}), \pi^2_i) = 1$$

for all $i \in [m]$.

Write

$$\mathsf{Adv}^{\mathsf{comb}}_{N,\mathcal{E}_3}(\mathcal{A}) := \Pr[\mathsf{Exp}^{\mathsf{comb}}_{N,\mathcal{E}_3,\mathcal{A}}(\lambda) = 1],$$

the probability taken as in Definition 16.

Lemma 20 (Combine tree unrolling)**.** *Assume Π_3 is knowledge-sound (Definition 3), and let $N \geq 2N_{\text{seg}}$ with $N_{\text{seg}} \mid N$ and $m := N/N_{\text{seg}}$. We construct an extractor $\mathcal{E}_3$ (given in the proof) such that for every PPT $\mathcal{A}$ there is a reduction adversary $\mathcal{D}_3$, running in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, with*

$$\mathsf{Adv}^{\mathsf{comb}}_{N,\mathcal{E}_3}(\mathcal{A}) \leq (m-1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}(\mathcal{D}_3).$$

Proof. The combine game returns 0 unless $\mathcal{A}$'s `combine` proof verifies, so we assume it does and bound the probability that extraction nonetheless violates the winning condition; write $m := N/N_{\text{seg}}$. The argument runs in four steps.

Step 1. *Recursion tree and bad events.* The extractor $\mathcal{E}_3$ unrolls a binary tree in a fixed depth-first order: at each `combine` node it runs $\mathcal{E}_{\Pi_3}$ on that node's statement and proof to obtain a candidate witness $(\pi^L, \pi^R, \hat{S}^{mid}, N_L, N_R)$, recurses into any child whose step count is at least $2N_{\text{seg}}$ (a `combine` child, by the typing of (20)), and stops at `convert` children ($N_{child} = N_{\text{seg}}$). For $t \geq 1$ define

$$B_t: \quad \text{the } \textit{first } \mathcal{E}_{\Pi_3} \text{ invocation (in unroll order) that does not return a valid } \mathcal{R}_3\text{-witness for its node's statement is invocation number } t.$$

The events B_t are disjoint, and at every B_t the invocation's proof verifies: the root's by the game's filter, a child's by its parent's valid witness. At the moment of the first failure every earlier invocation returned a valid witness; the step counts of a valid witness partition the node's count into multiples of N_{seg}, so the valid prefix contains at most $m-1$ internal nodes. Moreover, if all $m-1$ possible internal invocations are valid, every leaf-child has step count exactly N_{seg} and is a `convert` child by the typing of (20), so no further $\mathcal{E}_{\Pi_3}$ invocation exists. Hence only $B_1, \ldots, B_{m-1}$ are possible. Off all of them (i.e. if no invocation fails), the tree has exactly $m-1$ internal (`combine`) nodes and m leaves (`convert` proofs).

Step 2. *One reduction per invocation.* $\mathcal{D}_3^{(t)}$ runs $\mathcal{A}(1^\lambda)$ and unrolls the tree up to the t-th $\mathcal{E}_{\Pi_3}$ invocation (deterministic given $\mathcal{A}$'s output and the extractions at the earlier invocations), then forwards that invocation's statement and proof to the Π_3 knowledge-soundness challenger. It wins whenever B_t occurs, so

$$\Pr[B_t] \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}(\mathcal{D}_3^{(t)}).$$

Each $\mathcal{D}_3^{(t)}$ makes a single run of $\mathcal{A}$ and runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

Step 3. *Off the bad events, the extractor succeeds.* Assume $\neg B_1 \wedge \cdots \wedge \neg B_{m-1}$; every statement in this step is then deterministic. We argue by strong induction on N (justified by Remark 18).

(a) *Base case $N = 2N_{\text{seg}}$ ($m = 2$, one internal node).* Off B_1 (the root is the only $\mathcal{E}_{\Pi_3}$ invocation), applying $\mathcal{E}_{\Pi_3}$ to the root yields a valid witness with $N_L + N_R = 2N_{\text{seg}}$ and $N_{\text{seg}} \mid N_L$, $N_{\text{seg}} \mid N_R$, forcing $N_L = N_R = N_{\text{seg}}$. By the step-count typing of (20) both children are verified under V_2, i.e. they are the two leaf `convert` proofs; the extractor outputs them together with the boundary states $(\hat{S}_0, \hat{S}^{mid}, \hat{S}_N)$, meeting the winning condition. (No $\mathcal{R}_3$-witness exists at step count N_{seg} — the typing forces $N_L, N_R \ge N_{\text{seg}}$, hence $N_L + N_R \ge 2N_{\text{seg}}$ — which is why this lemma, like the system parameters of Definition 9, requires $N \ge 2N_{\text{seg}}$: a `combine` statement at step count N_{seg} is unsatisfiable, yet a proof for it could still *verify*, and no extractor could then meet the winning condition.)

(b) *Inductive step $N > 2N_{\text{seg}}$.* Let $\mathcal{A}$ output π^3 for $(\hat{S}_0, \hat{S}_N, N)$. Off B_1 (the root invocation), applying $\mathcal{E}_{\Pi_3}$ to $(\hat{S}_0, \hat{S}_N, N, \pi^3)$ yields a valid witness

$$(\pi^L, \pi^R, \hat{S}^{mid}, N_L, N_R)$$

for the statement $(\hat{S}_0, \hat{S}_N, N)$, i.e. with $N_L + N_R = N$, $N_{\text{seg}} \mid N_L$, $N_{\text{seg}} \mid N_R$, and each child verified according to its step count (20):

$$\begin{aligned}
&\big(N_L = N_{\text{seg}} \wedge \mathsf{V}_2((\hat{S}_0, \hat{S}^{mid}, N_L), \pi^L) = 1\big) \\
&\quad \vee \ \big(N_L \ge 2N_{\text{seg}} \wedge \mathsf{V}_3((\hat{S}_0, \hat{S}^{mid}, N_L), \pi^L) = 1\big), \\
&\big(N_R = N_{\text{seg}} \wedge \mathsf{V}_2((\hat{S}^{mid}, \hat{S}_N, N_R), \pi^R) = 1\big) \\
&\quad \vee \ \big(N_R \ge 2N_{\text{seg}} \wedge \mathsf{V}_3((\hat{S}^{mid}, \hat{S}_N, N_R), \pi^R) = 1\big).
\end{aligned}$$

Left child. If $N_L = N_{\text{seg}}$, the typing of (20) gives $\mathsf{V}_2(\cdot, \pi^L) = 1$, and π^L is a leaf `convert` proof $\pi^2_L := \pi^L$ covering $[\hat{S}_0, \hat{S}^{mid}]$. If instead $N_L \ge 2N_{\text{seg}}$, the typing gives $\mathsf{V}_3(\cdot, \pi^L) = 1$, so $2N_{\text{seg}} \le N_L < N$ and the left subtree is a smaller `combine` instance; its $\mathcal{E}_{\Pi_3}$ invocations are among those indexed by the $\{B_t\}$, so off those events the induction hypothesis (applied to the extractor $\mathcal{E}_3^{(N_L)}$ on $(\hat{S}_0, \hat{S}^{mid}, N_L, \pi^L)$) yields $m_L := N_L/N_{\text{seg}}$ `convert` proofs $\pi^2_1, \ldots, \pi^2_{m_L}$ and boundary states $\hat{S}^{(0)} = \hat{S}_0, \hat{S}^{(1)}, \ldots, \hat{S}^{(m_L)} = \hat{S}^{mid}$.

Right child. Symmetrically, the range $[\hat{S}^{mid}, \hat{S}_N]$ at step count $N_R < N$ yields $m_R := N_R/N_{\text{seg}}$ `convert` proofs.

Concatenate. Joining the two lists gives $m = m_L + m_R = N/N_{\text{seg}}$ `convert` proofs and a chain $\hat{S}^{(0)} = \hat{S}_0, \ldots, \hat{S}^{(m)} = \hat{S}_N$ with $\Pi_2.\mathsf{Verify}((\hat{S}^{(i-1)}, \hat{S}^{(i)}, N_{\text{seg}}), \pi^2_i) = 1$ for all $i \in [m]$ — exactly the winning condition of $\mathsf{Exp}^{\mathsf{comb}}$.

Step 4. *Conclusion (union bound).* Off all bad events the extractor meets the winning condition, so the game outputs 1 only if some B_t occurs, and only $B_1, \ldots, B_{m-1}$ are possible (Step 1), so by a union bound

$$\mathsf{Adv}^{\mathsf{comb}}_{N,\mathcal{E}_3}(\mathcal{A}) \;\le\; \sum_{t=1}^{m-1} \Pr[B_t] \;\le\; (m-1)\cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}(\mathcal{D}_3),$$

where $\mathcal{D}_3$ denotes the maximum-advantage reduction among $\mathcal{D}_3^{(1)}, \ldots, \mathcal{D}_3^{(m-1)}$ (the convention recorded in the "Advantage notation" paragraph). Since $m = N/N_{\mathrm{seg}}$ is polynomially bounded (Definition 9), the right-hand side is negligible whenever $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}$ is. The extractor invokes $\mathcal{E}_{\Pi_3}$ once per internal node, each call costing at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, so it runs in $(m-1)\cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

□

5.3.3 Convert layer: normalization

Definition 21 (Convert extraction game)**.** *For an extractor $\mathcal{E}_2$ and PPT $\mathcal{A}$, define the game $\mathsf{Exp}^{\mathsf{conv}}_{\mathcal{E}_2,\mathcal{A}}(\lambda)$:*

1. ***Adversary.*** *$\mathcal{A}(1^\lambda)$ outputs $(\hat{S}_{in}, \hat{S}_{out}, N_{\mathrm{seg}})$ and a proof π^2.*
2. ***Filter.*** *If $\Pi_2.\mathsf{Verify}((\hat{S}_{in}, \hat{S}_{out}, N_{\mathrm{seg}}), \pi^2) = 0$, return 0.*
3. ***Extraction.*** *Run $\pi^1 \leftarrow \mathcal{E}_2(\hat{S}_{in}, \hat{S}_{out}, N_{\mathrm{seg}}, \pi^2)$.*
4. ***Winning condition.*** *Return 1 iff $\Pi_1.\mathsf{Verify}((\hat{S}_{in}, \hat{S}_{out}), \pi^1) = 0$.*

Write $\mathsf{Adv}^{\mathsf{conv}}_{\mathcal{E}_2}(\mathcal{A}) := \Pr[\mathsf{Exp}^{\mathsf{conv}}_{\mathcal{E}_2,\mathcal{A}}(\lambda) = 1]$, the probability taken as in Definition 16.

Lemma 22 (Convert extraction)**.** *Assume Π_2 is knowledge-sound (Definition 3). We construct an extractor $\mathcal{E}_2$ (given in the proof) such that for every PPT $\mathcal{A}$ there is a reduction adversary $\mathcal{D}_2$, running in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, with*

$$\mathsf{Adv}^{\mathsf{conv}}_{\mathcal{E}_2}(\mathcal{A}) \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_2}(\mathcal{D}_2).$$

Proof. Set $\mathcal{E}_2 := \mathcal{E}_{\Pi_2}$ (the $\mathcal{R}_2$ witness is exactly π^1), and let $\mathcal{D}_2$ run $\mathcal{A}$ and forward

$$((\hat{S}_{in}, \hat{S}_{out}, N_{\mathrm{seg}}), \pi^2)$$

to its challenger. The convert game returns 1 exactly when π^2 verifies but

$$\Pi_1.\mathsf{Verify}((\hat{S}_{in}, \hat{S}_{out}), \pi^1) = 0;$$

this implies $(x; \pi^1) \notin \mathcal{R}_2$, the event that $\mathcal{D}_2$ wins the knowledge-soundness game of Π_2, so $\mathsf{Adv}^{\mathsf{conv}}_{\mathcal{E}_2}(\mathcal{A}) \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_2}(\mathcal{D}_2)$. (Equality can fail when $\mathcal{A}$'s statement carries a step count other than N_{seg}: the statement is then outside the support of $\mathcal{R}_2$, so $\mathcal{D}_2$ wins whenever π^2 verifies, while the convert game may still return 0.) □

5.3.4 Segment layer: inner proofs to a segment

Definition 23 (Segment extraction game)**.** *For an extractor $\mathcal{E}_1$ and PPT $\mathcal{A}$, define the game $\mathsf{Exp}^{\mathsf{seg}}_{\mathcal{E}_1,\mathcal{A}}(\lambda)$:*

1. ***Adversary.*** *$\mathcal{A}(1^\lambda)$ outputs $(\hat{S}_{in}, \hat{S}_{out})$ and a proof π^1.*
2. ***Filter.*** *If $\Pi_1.\mathsf{Verify}((\hat{S}_{in}, \hat{S}_{out}), \pi^1) = 0$, return 0.*
3. ***Extraction.*** *Run $(\widehat{\mathcal{B}}, \hat{S}_1, \ldots, \hat{S}_{N_{\text{seg}}+1}, \{\pi_i^{\mathsf{mem}}\}_{i=1}^{N_{\text{seg}}}) \leftarrow \mathcal{E}_1(\hat{S}_{in}, \hat{S}_{out}, \pi^1)$.*
4. ***Winning condition.*** *Return 1 unless $\hat{S}_1 = \hat{S}_{in}$, $\hat{S}_{N_{\text{seg}}+1} = \hat{S}_{out}$, and*

$$\bigwedge_{i=1}^{N_{\text{seg}}} \widehat{\varphi}_{\text{step}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi_i^{\mathsf{mem}}) = \text{TRUE}.$$

Write $\mathsf{Adv}^{\mathsf{seg}}_{\mathcal{E}_1}(\mathcal{A}) := \Pr[\mathsf{Exp}^{\mathsf{seg}}_{\mathcal{E}_1,\mathcal{A}}(\lambda) = 1]$, the probability taken as in Definition 16.

Lemma 24 (Segment extraction)**.** *Assume Π_1 and each $\Pi_{0,j}$ ($j \in \{\text{step}, \text{keccak}, \text{poseidon}, \text{range}\}$) are knowledge-sound (Definition 3), and $\text{Com}_{\mathsf{bus}}$ is collision-resistant (Definition 7). We construct an extractor $\mathcal{E}_1$ (given in the proof) such that for every PPT $\mathcal{A}$ there are reduction adversaries $\mathcal{D}_1, \mathcal{D}_{0,\text{step}}, \mathcal{D}_{0,\text{keccak}}, \mathcal{D}_{0,\text{poseidon}}, \mathcal{D}_{0,\text{range}}, \mathcal{D}_{\mathsf{bus}}$, each running in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$, with*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{seg}}_{\mathcal{E}_1}(\mathcal{A}) \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_1}(\mathcal{D}_1) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{step}}}(\mathcal{D}_{0,\text{step}}) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{keccak}}}(\mathcal{D}_{0,\text{keccak}}) \\ & + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{poseidon}}}(\mathcal{D}_{0,\text{poseidon}}) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{range}}}(\mathcal{D}_{0,\text{range}}) + \mathsf{Adv}^{\mathsf{cr}}_{\text{Com}_{\mathsf{bus}}}(\mathcal{D}_{\mathsf{bus}}).\end{aligned}$$

Proof. The segment game returns 0 unless π^1 verifies, so we assume it does and bound the probability that extraction violates the winning condition. We define six bad events, one per reduction, and show that off all six the extractor meets the winning condition; a union bound then gives the stated sum.

Bad events and their reductions. The six events below refer to the intermediate values produced by the corresponding steps of the extractor $\mathcal{E}_1$, defined in the next paragraph; on any fixed run, read the extractor's steps first and interpret each event as "that step's extraction fails". Each reduction runs $\mathcal{A}(1^\lambda)$ once (invoking the earlier partial extractors as a subroutine) and runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.

- B_1 (Π_1 *extraction*): π^1 verifies but $\mathcal{E}_{\Pi_1}$ returns a witness $(C_{\widehat{\mathcal{B}}}, \pi_{\text{step}}, \pi_{\text{keccak}}, \pi_{\text{poseidon}}, \pi_{\text{range}}) \notin \mathcal{R}_1$. $\mathcal{D}_1$ forwards $((\hat{S}_{in}, \hat{S}_{out}), \pi^1)$ to the Π_1 challenger, so $\Pr[B_1] \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_1}(\mathcal{D}_1)$.
- $B_{0,\text{step}}$ (*inner-step extraction*): π_{step} verifies but $\mathcal{E}_{0,\text{step}}$ returns a witness $\notin \mathcal{R}_{0,\text{step}}$. Here $\mathcal{D}_{0,\text{step}}$ is the partial extractor running Step 1, forwarding $(x_{0,\text{step}}, \pi_{\text{step}})$, so that $\Pr[B_{0,\text{step}}]$ is at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{step}}}(\mathcal{D}_{0,\text{step}})$.
- $B_{0,j}$ for $j \in \{\text{keccak}, \text{poseidon}, \text{range}\}$ (*inner-chip extraction*): π_j verifies but $\mathcal{E}_{0,j}$ returns a witness $\notin \mathcal{R}_{0,j}$. $\mathcal{D}_{0,j}$ is the partial extractor running Steps 1–2, forwarding $(x_{0,j}, \pi_j)$; so $\Pr[B_{0,j}] \le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,j}}(\mathcal{D}_{0,j})$.
- B_{bus} (*bus collision*): the extracted buses $\widehat{\mathcal{B}}_{\text{step}}, \widehat{\mathcal{B}}_{\text{keccak}}, \widehat{\mathcal{B}}_{\text{poseidon}}, \widehat{\mathcal{B}}_{\text{range}}$ are not all equal, although all four commit to $C_{\widehat{\mathcal{B}}}$. $\mathcal{D}_{\mathsf{bus}}$ outputs any unequal pair as a $\text{Com}_{\mathsf{bus}}$ collision, so $\Pr[B_{\mathsf{bus}}] \le \mathsf{Adv}^{\mathsf{cr}}_{\text{Com}_{\mathsf{bus}}}(\mathcal{D}_{\mathsf{bus}})$.

The extractor $\mathcal{E}_1$ (six steps). Step 1. Apply $\mathcal{E}_{\Pi_1}$ to $(\hat{S}_{in}, \hat{S}_{out}, \pi^1)$ to obtain

$$(C_{\hat{\mathcal{B}}}, \pi_{\text{step}}, \pi_{\text{keccak}}, \pi_{\text{poseidon}}, \pi_{\text{range}});$$

off B_1 these satisfy

$$\Pi_{0,j}.\mathsf{Verify}(x_{0,j}, \pi_j) = 1 \quad \text{for all } j \in \{\text{step}, \text{keccak}, \text{poseidon}, \text{range}\},$$

with $x_{0,\text{step}} = (\hat{S}_{in}, \hat{S}_{out}, C_{\hat{\mathcal{B}}})$ and $x_{0,j} = C_{\hat{\mathcal{B}}}$ otherwise.

Step 2. Apply $\mathcal{E}_{0,\text{step}}$ (Lemma 26) to $(x_{0,\text{step}}, \pi_{\text{step}})$; off $B_{0,\text{step}}$ obtain $(\widehat{\mathcal{B}}_{\text{step}}, \{\hat{S}_i\}, \{\pi_i^{\mathsf{mem}}\})$ with $\hat{S}_1 = \hat{S}_{in}$, $\hat{S}_{N_{\text{seg}}+1} = \hat{S}_{out}$,

$$\textstyle\bigwedge_i \widehat{\varphi}_{\text{step,bus}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}_{\text{step}}, \pi_i^{\mathsf{mem}}) = \text{TRUE},$$

and $\text{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}_{\text{step}}) = C_{\hat{\mathcal{B}}}$.

Step 3. Apply $\mathcal{E}_{0,j}$ to $(x_{0,j}, \pi_j)$ for $j \in \{\text{keccak}, \text{poseidon}, \text{range}\}$; off $B_{0,j}$ obtain buses $\widehat{\mathcal{B}}_{\text{keccak}}, \widehat{\mathcal{B}}_{\text{poseidon}}, \widehat{\mathcal{B}}_{\text{range}}$, each satisfying its chip predicate and committing to $C_{\hat{\mathcal{B}}}$.

Step 4. Set $\widehat{\mathcal{B}} := \widehat{\mathcal{B}}_{\text{step}}$.

Output. $\mathcal{E}_1$ returns $\widehat{\mathcal{B}}$, the states $\hat{S}_1, \ldots, \hat{S}_{N_{\text{seg}}+1}$ of Step 2, and the memory-opening witnesses $\{\pi_i^{\mathsf{mem}}\}_{i=1}^{N_{\text{seg}}}$ (entries of the $\mathcal{R}_{0,\text{step}}$ witness).

Off all six events, the extractor wins. Assume $\neg B_1 \wedge \neg B_{0,\text{step}} \wedge \bigwedge_j \neg B_{0,j} \wedge \neg B_{\mathsf{bus}}$.

Bus consistency. All four buses commit to $C_{\hat{\mathcal{B}}}$, and off B_{bus} they are equal; this is consistent with the choice $\widehat{\mathcal{B}} := \widehat{\mathcal{B}}_{\text{step}}$.

Boundary conditions. Step 2 gives $\hat{S}_1 = \hat{S}_{in}$ and $\hat{S}_{N_{\text{seg}}+1} = \hat{S}_{out}$ directly from the $\mathcal{R}_{0,\text{step}}$ witness.

Step predicate. Combining Step 2 ($\widehat{\varphi}_{\text{step,bus}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi_i^{\mathsf{mem}}) = \text{TRUE}$ for all i) with Step 3 ($\widehat{\varphi}_{\text{keccak}}(\widehat{\mathcal{B}}) = \widehat{\varphi}_{\text{poseidon}}(\widehat{\mathcal{B}}) = \widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) = \text{TRUE}$), Eq. (14) gives

$$\begin{aligned}\widehat{\varphi}_{\text{step}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi_i^{\mathsf{mem}}) \;=\; & \widehat{\varphi}_{\text{step,bus}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi_i^{\mathsf{mem}}) \\ & \wedge\; \widehat{\varphi}_{\text{keccak}}(\widehat{\mathcal{B}}) \;\wedge\; \widehat{\varphi}_{\text{poseidon}}(\widehat{\mathcal{B}}) \;\wedge\; \widehat{\varphi}_{\text{range}}(\widehat{\mathcal{B}}) \;=\; \text{TRUE}\end{aligned}$$

for all $i \in \{1, \ldots, N_{\text{seg}}\}$. Together with the boundary conditions this is exactly the winning condition of $\mathsf{Exp}^{\mathsf{seg}}$.

Conclusion. Off all six bad events the game outputs 0, so it outputs 1 only if one of them occurs. By a union bound,

$$\begin{aligned}\mathsf{Adv}^{\mathsf{seg}}_{\mathcal{E}_1}(\mathcal{A}) &\le \Pr[B_1] + \Pr[B_{0,\text{step}}] + \sum_{j \in \{\text{keccak,poseidon,range}\}} \Pr[B_{0,j}] + \Pr[B_{\mathsf{bus}}] \\ &\le \mathsf{Adv}^{\mathsf{ks}}_{\Pi_1}(\mathcal{D}_1) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{step}}}(\mathcal{D}_{0,\text{step}}) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{keccak}}}(\mathcal{D}_{0,\text{keccak}}) \\ &\quad + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{poseidon}}}(\mathcal{D}_{0,\text{poseidon}}) + \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,\text{range}}}(\mathcal{D}_{0,\text{range}}) + \mathsf{Adv}^{\mathsf{cr}}_{\text{Com}_{\mathsf{bus}}}(\mathcal{D}_{\mathsf{bus}}),\end{aligned}$$

as claimed. □

5.3.5 Inner-circuit layer

Definition 25 (Inner-circuit extraction game)**.** *For $j \in \{\text{step}, \text{keccak}, \text{poseidon}, \text{range}\}$, an extractor $\mathcal{E}_{0,j}$, and PPT $\mathcal{A}$, define the game $\mathsf{Exp}^{\mathsf{inner}}_{j,\mathcal{E}_{0,j},\mathcal{A}}(\lambda)$:*

1. ***Adversary.*** *$\mathcal{A}(1^\lambda)$ outputs a public input $x_{0,j}$ and a proof $\pi_{0,j}$.*
2. ***Filter.*** *If $\Pi_{0,j}.\mathsf{Verify}(x_{0,j}, \pi_{0,j}) = 0$, return 0.*
3. ***Extraction.*** *Run $w_{0,j} \leftarrow \mathcal{E}_{0,j}(x_{0,j}, \pi_{0,j})$.*
4. ***Winning condition.*** *Return 1 iff $(x_{0,j}; w_{0,j}) \notin \mathcal{R}_{0,j}$.*

Write $\mathsf{Adv}^{\mathsf{inner}}_{j,\mathcal{E}_{0,j}}(\mathcal{A}) := \Pr[\mathsf{Exp}^{\mathsf{inner}}_{j,\mathcal{E}_{0,j},\mathcal{A}}(\lambda) = 1]$, the probability taken as in Definition 16; this is precisely the knowledge-soundness game of $\Pi_{0,j}$.

Lemma 26 (Inner-circuit extraction)**.** *Assume each $\Pi_{0,j}$, $j \in \{\text{step}, \text{keccak}, \text{poseidon}, \text{range}\}$, is knowledge-sound (Definition 3). For each j we take the extractor $\mathcal{E}_{0,j} := \mathcal{E}_{\Pi_{0,j}}$ such that for every PPT $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{inner}}_{j,\mathcal{E}_{0,j}}(\mathcal{A}) = \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,j}}(\mathcal{D}_{0,j}), \qquad \mathcal{D}_{0,j} := \mathcal{A},$$

the game being that of knowledge soundness. The extracted witness $w_{0,j}$ has the following structure:

- *$j = \text{step}$: $w_{0,\text{step}} = (\widehat{\mathcal{B}}, \{\hat{S}_i\}_{1 \le i \le N_{\text{seg}}+1}, \{\pi^{\mathsf{mem}}_i\}_{1 \le i \le N_{\text{seg}}})$ with $\hat{S}_1 = \hat{S}_{in}$, $\hat{S}_{N_{\text{seg}}+1} = \hat{S}_{out}$,*

$$\bigwedge_{i=1}^{N_{\text{seg}}} \widehat{\varphi}_{\text{step,bus}}(\hat{S}_i, \hat{S}_{i+1}, \widehat{\mathcal{B}}, \pi^{\mathsf{mem}}_i) = \text{TRUE},$$

  and $\mathrm{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) = C_{\hat{\mathcal{B}}}$.

- *$j \in \{\text{keccak}, \text{poseidon}, \text{range}\}$: $w_{0,j} = \widehat{\mathcal{B}}$ with $\widehat{\varphi}_j(\widehat{\mathcal{B}}) = \text{TRUE}$ and $\mathrm{Com}_{\mathsf{bus}}(\widehat{\mathcal{B}}) = C_{\hat{\mathcal{B}}}$.*

Proof. $\mathcal{E}_{0,j} = \mathcal{E}_{\Pi_{0,j}}$ is the knowledge-soundness extractor of $\Pi_{0,j}$, so $\mathsf{Exp}^{\mathsf{inner}}_j$ *is* its knowledge-soundness game and the advantages coincide; the extracted witness lies in $\mathcal{R}_{0,j}$, giving the structure listed above. □

5.4 Main theorem

Theorem 27 (Correct-trace extractability)**.** *Let zkVM be a zkVM system (Definition 8) with step count T a system parameter satisfying $N_{\text{seg}} \mid T$, $T \ge 2N_{\text{seg}}$, and $T \le \mathsf{poly}(\lambda)$ (as in Definition 9), and let $m := T/N_{\text{seg}}$. For every PPT adversary $\mathcal{A}$ with CTE advantage $\varepsilon := \mathsf{Adv}^{\mathsf{cte}}_{\mathsf{zkVM}}(\mathcal{A})$ there exist PPT reduction adversaries $\mathcal{D}_4$, $\mathcal{D}_3$, $\mathcal{D}_2$, $\mathcal{D}_1$, $\{\mathcal{D}_{0,j}\}_j$, $\mathcal{D}_{\mathsf{bus}}$, $\{\mathcal{D}^{\mathsf{pos}}_k\}_{k=1}^T$, $\{\mathcal{D}^{\mathsf{upd}}_k\}_{k=1}^T$, $\mathcal{D}^{\mathsf{pos}}_{\mathsf{fin}}$ such that*

$$\begin{aligned}
\varepsilon \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_4}(\mathcal{D}_4) + (m-1)\,\mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}(\mathcal{D}_3) + m\,\mathsf{Adv}^{\mathsf{ks}}_{\Pi_2}(\mathcal{D}_2) \\
& + m\Big(\mathsf{Adv}^{\mathsf{ks}}_{\Pi_1}(\mathcal{D}_1) + \sum_{j \in \{\text{step,keccak,poseidon,range}\}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,j}}(\mathcal{D}_{0,j}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathrm{Com}_{\mathsf{bus}}}(\mathcal{D}_{\mathsf{bus}})\Big) \\
& + \sum_{k=1}^{T}\Big(\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}_k) + \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}}_k)\Big) + \mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}_{\mathsf{fin}}),
\end{aligned} \tag{24}$$

where the sum over j runs over $\{\text{step}, \text{keccak}, \text{poseidon}, \text{range}\}$ *and k from* 1 *to T; the final summand accounts for the boundary check* $\mathtt{mem}'_T = \mathtt{mem}_T$*, a bad event distinct from the T per-step ones (Step 6 of the proof). All reduction adversaries invoke $\mathcal{A}$ exactly once and run in time at most $c \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ with $c = 7m + 1$ (computed in the running-time paragraph at the end of the proof). In particular, if all advantage terms on the right-hand side are negligible in λ, then* zkVM *is correct-trace extractable (Definition 11).*

Proof. Let $\mathcal{A}$ be a PPT adversary with CTE advantage ε (Definition 11); we bound ε. We construct $\mathcal{E}_{\mathsf{zkVM}}$ by composing the sub-extractors guaranteed by Lemmas 17, 20, 22, 24, and 26. Each sub-extractor is straight-line and is applied sequentially to the proof extracted by the preceding layer.

Step 1 (Embed layer). Apply $\mathcal{E}_4$ (Lemma 17) to $(\hat{S}_0, \hat{S}_T, \pi^4)$ to obtain π^3 with

$$\Pi_3.\mathsf{Verify}((\hat{S}_0, \hat{S}_T, T), \pi^3) = 1$$

. Fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_4}(\mathcal{D}_4)$, where $\mathcal{D}_4$ runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$.[7]

Step 2 (Combine layer). Apply $\mathcal{E}_3$ (Lemma 20) to $(\hat{S}_0, \hat{S}_T, T, \pi^3)$ to obtain $m = T/N_{\text{seg}}$ convert proofs $\pi^2_1, \ldots, \pi^2_m$ and boundary states

$$\hat{S}^{(0)} = \hat{S}_0, \hat{S}^{(1)}, \ldots, \hat{S}^{(m)} = \hat{S}_T$$

. Fails with probability at most

$$(m-1) \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}(\mathcal{D}_3)$$

, where $\mathcal{D}_3$ runs in $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ and the extractor $\mathcal{E}_3$ in

$$(m-1) \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$$

.

Step 3 (Convert layer, applied m times). For each $i \in [m]$, apply $\mathcal{E}_2$ (Lemma 22) to $(\hat{S}^{(i-1)}, \hat{S}^{(i)}, N_{\text{seg}}, \pi^2_i)$. Obtain π^1_i with $\Pi_1.\mathsf{Verify}((\hat{S}^{(i-1)}, \hat{S}^{(i)}), \pi^1_i) = 1$. Each invocation i fails with probability at most $\mathsf{Adv}^{\mathsf{ks}}_{\Pi_2}(\mathcal{D}_2)$; the total over m calls is $m \cdot \mathsf{Adv}^{\mathsf{ks}}_{\Pi_2}(\mathcal{D}_2)$.

Step 4 (Segment layer, applied m times). For each $i \in [m]$, apply $\mathcal{E}_1$ (Lemma 24, which internally uses Lemma 26 and the collision resistance of $\mathrm{Com}_{\mathsf{bus}}$ (Definition 7)) to obtain a bus $\widehat{\mathcal{B}}_i$ and states $\hat{S}^{(i,0)}, \ldots, \hat{S}^{(i,N_{\text{seg}})}$ satisfying

$$\hat{S}^{(i,0)} = \hat{S}^{(i-1)}, \quad \hat{S}^{(i,N_{\text{seg}})} = \hat{S}^{(i)}, \quad \bigwedge_{j=1}^{N_{\text{seg}}} \widehat{\varphi}_{\text{step}}(\hat{S}^{(i,j-1)}, \hat{S}^{(i,j)}, \widehat{\mathcal{B}}_i) = \text{TRUE}.$$

(The step predicates are evaluated on the extracted memory-opening witnesses π^{mem}, per the shorthand convention of Section 5.2.) Each invocation i fails with probability at most

$$\mathsf{Adv}^{\mathsf{ks}}_{\Pi_1}(\mathcal{D}_1) + \sum_{j \in \{\text{step},\text{keccak},\text{poseidon},\text{range}\}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,j}}(\mathcal{D}_{0,j}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathrm{Com}_{\mathsf{bus}}}(\mathcal{D}_{\mathsf{bus}});$$

the total over m calls is m times that sum.

Step 5 (Concatenation). Define $\hat{S}_k := \hat{S}^{(\lfloor k/N_{\text{seg}} \rfloor + 1,\ k \bmod N_{\text{seg}})}$ for $k = 0, \ldots, T-1$, and $\hat{S}_T := \hat{S}^{(m,N_{\text{seg}})}$. The boundary conditions $\hat{S}^{(i,N_{\text{seg}})} = \hat{S}^{(i)} = \hat{S}^{(i+1,0)}$ ensure the chain is consistent

[7] This composition is exactly where the over-idealization of Remark 2 and the "Straight-line extraction" remark is invoked: for succinct Π_4 a straight-line extractor from (x, π) alone cannot exist, and composing straight-line extraction across the recursion would require relativized SNARKs, which do not exist in the ROM. We idealize over both, as declared there.

across segment boundaries. We obtain a sequence $\hat{S}_0, \hat{S}_1, \ldots, \hat{S}_T$ of committed-memory states with $\widehat{\varphi}_{\text{step}}(\hat{S}_k, \hat{S}_{k+1}, \widehat{\mathcal{B}}_{\lceil (k+1)/N_{\text{seg}} \rceil}) = \text{TRUE}$ for all $k \in \{0, \ldots, T-1\}$.

Step 6 (Memory reconstruction and memory extractability). Recall that CTE (Definition 11) requires the extractor to output a witness for $\mathcal{R}^*$ (22), i.e. full-memory states $S_0, \ldots, S_T$ with $\mathtt{mem}'_0 = \mathtt{mem}_0$, $\mathtt{mem}'_T = \mathtt{mem}_T$, and $\varphi_{\text{step}}(S_k, S_{k+1}) = \text{TRUE}$ for all k. Since the extractor receives the statement (S_0, S_T), hence the full boundary states (Remark 12, Step 3), the initial memory $\mathtt{mem}_0$ is given directly, so $\mathtt{mem}'_0 = \mathtt{mem}_0$; the initial committed state satisfies $\hat{S}_0.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_0)$ by hypothesis. For each step $k \in \{0, \ldots, T-1\}$, Lemma 24 additionally yields the memory-opening witness π_k^{mem} for that step; on a read it is parsed as the opening proof π required by $\widehat{\varphi}_{\text{read}}$ (12), on a write as the pair $(\pi_{\text{write}}, v_{\mathsf{old}})$ required by $\widehat{\varphi}_{\text{write}}$ (13), and on any other step it is left uninterpreted. The memory vectors are then reconstructed by the rule (23): $\mathtt{mem}_{k+1} := \mathtt{mem}_k[\mathtt{addr} \mapsto x]$ with $\mathtt{addr} := \hat{S}_k.\mathtt{regs}[0]$, $x := \hat{S}_k.\mathtt{regs}[1]$ on a write step, and $\mathtt{mem}_{k+1} := \mathtt{mem}_k$ otherwise.

We proceed by induction on k, carrying the commitment invariant $\hat{S}_k.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k)$. It holds at $k = 0$ as noted above. For the inductive step, we verify that all requirements of Proposition 13 are satisfied at step k:

1. **Commitment invariant at k.** $\hat{S}_k.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_k)$ is the induction hypothesis. Note that the proposition requires the invariant only at the *first* of the two states; the invariant at $k+1$ is one of its conclusions, and is what carries the induction forward.

2. **Step predicate.** From Step 5,
$$\widehat{\varphi}_{\text{step}}(\hat{S}_k, \hat{S}_{k+1}, \widehat{\mathcal{B}}_{\lceil (k+1)/N_{\text{seg}} \rceil}) = \text{TRUE}.$$

3. **Opening witnesses (read case).** If step k is a read, the witness π parsed from π_k^{mem} satisfies $\hat{S}_k.\mathtt{mem} = \hat{S}_{k+1}.\mathtt{mem}$ and
$$\mathsf{Verify}(\hat{S}_k.\mathtt{mem}, \hat{S}_k.\mathtt{regs}[0], \hat{S}_{k+1}.\mathtt{regs}[1], \pi) = 1,$$
as required by the read requirement of the proposition.

4. **Opening witnesses (write case).** If step k is a write, the witnesses $(\pi_{\text{write}}, v_{\mathsf{old}})$ parsed from π_k^{mem} satisfy
$$\mathsf{Verify}(\hat{S}_k.\mathtt{mem}, \hat{S}_k.\mathtt{regs}[0], v_{\mathsf{old}}, \pi_{\text{write}}) = 1$$
and
$$\mathsf{Verify}(\hat{S}_{k+1}.\mathtt{mem}, \hat{S}_k.\mathtt{regs}[0], \hat{S}_k.\mathtt{regs}[1], \pi_{\text{write}}) = 1,$$
as required by the write requirement of the proposition.

Applying Proposition 13 with the adversary $\mathcal{A}_k$ defined as the composed extractor from Steps 1–5 that outputs
$$(\hat{S}_k, \hat{S}_{k+1}, \mathtt{mem}_k, \widehat{\mathcal{B}}_{\lceil (k+1)/N_{\text{seg}} \rceil}, w_k),$$
where w_k is the memory-opening witness π_k^{mem} parsed above, yields both conclusions for $S_k := (\hat{S}_k.\mathtt{pc}, \hat{S}_k.\mathtt{regs}, \mathtt{mem}_k)$:
$$\hat{S}_{k+1}.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_{k+1}) \qquad \text{and} \qquad \varphi_{\text{step}}(S_k, S_{k+1}) = \text{TRUE}.$$
The first re-establishes the invariant at $k+1$ and completes the induction; the second is the per-step obligation of (22). Both are obtained from the same pair of binding events, so the invariant is not paid for separately. The proposition invokes position-binding (for the read and write cases) and

update-binding (for the write case) of $\mathrm{Com}_{\mathsf{mem}}$; if either property is violated at step k this sub-step fails. There are T such steps, each contributing at most

$$\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}_k) + \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}}_k)$$

to the failure probability. Finally, the boundary memories match the claimed states: the extractor receives S_0, so $\mathtt{mem}'_0 = \mathtt{mem}_0$ by construction, and for the final memory the reconstruction gives $\hat{S}_T.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}'_T)$ while by hypothesis $\hat{S}_T.\mathtt{mem} = \mathsf{Commit}(\mathtt{mem}_T)$; if $\mathtt{mem}'_T \neq \mathtt{mem}_T$, honest openings at a differing index verify against the common root and break position-binding, so $\mathtt{mem}'_T = \mathtt{mem}_T$ except with probability

$$\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}_{\mathsf{fin}}).$$

This is a distinct bad event from the T per-step ones — it compares the reconstructed $\mathtt{mem}'_T$ against the adversary's claimed $\mathtt{mem}_T$, which no per-step reduction ever inspects — and $\mathcal{D}^{\mathsf{pos}}_{\mathsf{fin}}$ is a correspondingly distinct reduction, which computes the two honest openings at a differing index itself. It therefore contributes its own summand to (24) and is *not* absorbed into the $\sum_k$ term. At segment boundaries, $\hat{S}^{(i,N_{\mathrm{seg}})} = \hat{S}^{(i+1,0)}$ implies $\mathsf{Commit}(\mathtt{mem}_{N_{\mathrm{seg}}\cdot i})$ is the same on both sides, so the memory is consistent.

Conclusion. Summing all failure contributions across Steps 1–6 gives exactly the right-hand side of (24):

$$\begin{aligned}
\Pr[\mathrm{fail}] \le\ & \mathsf{Adv}^{\mathsf{ks}}_{\Pi_4}(\mathcal{D}_4) && \text{(Step 1)}\\
&+ (m{-}1)\,\mathsf{Adv}^{\mathsf{ks}}_{\Pi_3}(\mathcal{D}_3) && \text{(Step 2)}\\
&+ m\,\mathsf{Adv}^{\mathsf{ks}}_{\Pi_2}(\mathcal{D}_2) && \text{(Step 3)}\\
&+ m\left(\mathsf{Adv}^{\mathsf{ks}}_{\Pi_1}(\mathcal{D}_1) + \sum_{j\in\{\mathrm{step,keccak,poseidon,range}\}} \mathsf{Adv}^{\mathsf{ks}}_{\Pi_{0,j}}(\mathcal{D}_{0,j}) + \mathsf{Adv}^{\mathsf{cr}}_{\mathrm{Com}_{\mathsf{bus}}}(\mathcal{D}_{\mathsf{bus}})\right) && \text{(Step 4)}\\
&+ \sum_{k=1}^{T}\Big(\mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}_k) + \mathsf{Adv}^{\mathsf{upd}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{upd}}_k)\Big) + \mathsf{Adv}^{\mathsf{pos}}_{\mathrm{Com}_{\mathsf{mem}}}(\mathcal{D}^{\mathsf{pos}}_{\mathsf{fin}}). && \text{(Step 6)}
\end{aligned}$$

All coefficients $m = T/N_{\mathrm{seg}}$ and T are polynomially bounded in λ, so the right-hand side is a polynomially-sized sum of security advantages and is negligible whenever each advantage term is.

Running time. $\mathcal{E}_{\mathsf{zkVM}}$ never runs $\mathcal{A}$; its cost is the straight-line sub-extractor calls plus deterministic work. Steps 1–3 make $1 + (m-1) + m = 2m$ extractor calls ($\mathcal{E}_{\Pi_4}$, one $\mathcal{E}_{\Pi_3}$ per internal node, one $\mathcal{E}_{\Pi_2}$ per segment); Step 4 makes $5m$ ($\mathcal{E}_{\Pi_1}$ and the four inner extractors, per segment); Steps 5–6 are deterministic — indexing and the memory reconstruction rule (23) — costing $\mathsf{poly}(\lambda)$ and invoking no extractor. Under the convention that each sub-extractor call costs at most $\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$ (the "Advantage notation" paragraph), $\mathsf{Time}(\mathcal{E}_{\mathsf{zkVM}}) \le 7m \cdot \mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$. Each reduction adversary runs $\mathcal{A}$ once and re-runs the extraction chain above its own layer, hence runs in at most $(7m+1)\cdot\mathsf{Time}(\mathcal{A}) + \mathsf{poly}(\lambda)$; this is the constant $c = 7m+1$ of the statement. (Strictly, the convention of Definition 3 bounds each extractor by the time of the *composed* adversary that produced its input, which compounds along the recursion to $O(m^2)\cdot\mathsf{Time}(\mathcal{A})$ in the worst case; the linear figure uses the per-call convention and is part of the idealization of Remark 2.) In particular, $\mathcal{E}_{\mathsf{zkVM}}$ is PPT whenever $\mathcal{A}$ is PPT. □

References

[1] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. Relativized Succinct Arguments in the ROM Do Not Exist. Cryptology ePrint Archive, Paper 2024/728, 2024. https://eprint.iacr.org/2024/728. (TCC 2025.)